// project documentation

k0lmenIA

QA agents for Claude Code: test design, automation with k0lmena and integration with Xray, QMetry, AIO Tests and Azure DevOps. The agents work in English.

76 agents

Web · API · Mobile · Performance

Playwright · Cucumber · Appium · k6 · Artillery · JMeter

Xray · QMetry · AIO Tests · Azure DevOps

Jira · Confluence · Figma

GitHub · GitLab · Bitbucket

Linear · ClickUp · monday · Asana · Trello · Notion

Slack · Teams

PostgreSQL · MySQL · SQL Server · MongoDB

Token-free suites

Contents

1 What is k0lmenIA

2 Architecture

3 How to use it

4 Agents

5 Requirements

6 Installation

7 Configuration: the .env file

8 Project documentation

9 Context research and missing information

10 Task guide: design and execution

11 Automation with k0lmena

12 Performance testing

13 Test management: Xray, QMetry, AIO Tests and Azure DevOps

14 Database checks

15 Exploratory testing

16 Accessibility

17 Web performance

18 Failure analysis

19 Server logs

20 Coverage and traceability

21 Visual regression (pixel perfect)

22 Cross-browser

23 Repairing the automation

24 Selecting cases for a change

25 Case review and estimated time

26 Charters and exploratory sessions

27 Emails

28 Translations and languages

29 Usability

30 Forms

31 Role permissions

32 Resilience

33 Bug triage and questions

34 Run history

35 Quality dashboard and report portal

36 Notifications and synthetic monitoring

37 Agent evals

38 Token usage

39 Onboarding

40 Web security

41 Streaming and cameras

42 MCP connectors

43 Conventions

44 Repository structure

45 Troubleshooting

46 Extending the project

// introduction

What is k0lmenIA

A team of **specialized QA agents** for Claude Code, built for manual testing professionals: no programming needed, you work by talking to it inside VS Code.

You provide an input (a user story, a bug observation, an API contract, or simply a Jira key or a Figma link), ask for what you need in natural language, and the right agent produces the result: analyses, plans, test cases in Excel or Gherkin, test data, bug reports, HTML reports, automation and upload to your test management tool. The agents and their deliverables are in English.

1 · Design

Analyze stories, plan, write and review manual, BDD and API test cases (with their estimated time), generate data and charters, and write bug reports.

Saved in `output/`

2 · Automation

The *mapper* agents write the automation once; after that it runs with `npm`, with no tokens.

Saved in `tools/k0lmena/`

3 · Management

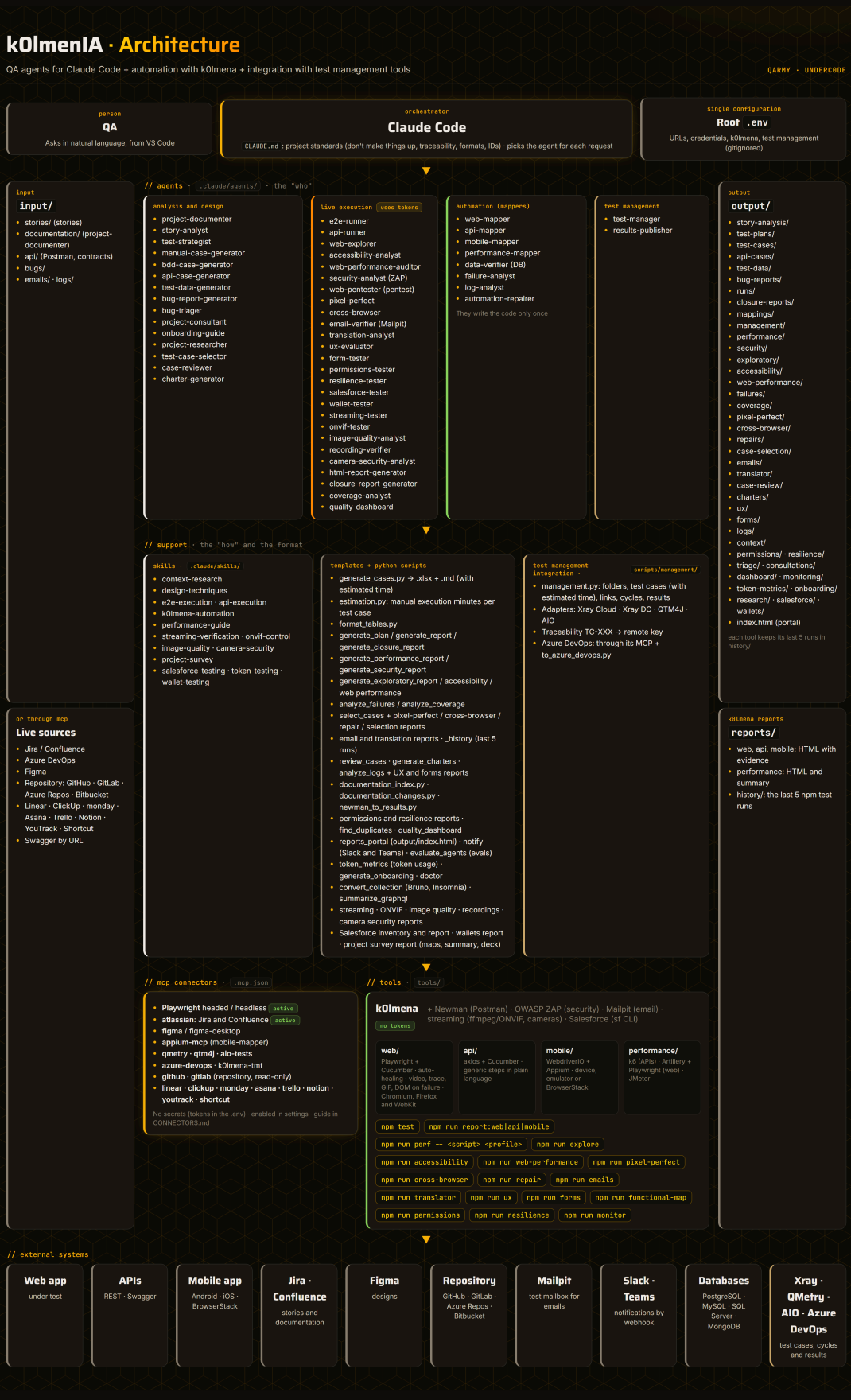
Test cases, cycles and results with evidence in Xray, QMetry or AIO Tests (test cases also in Azure DevOps), and the closing report.

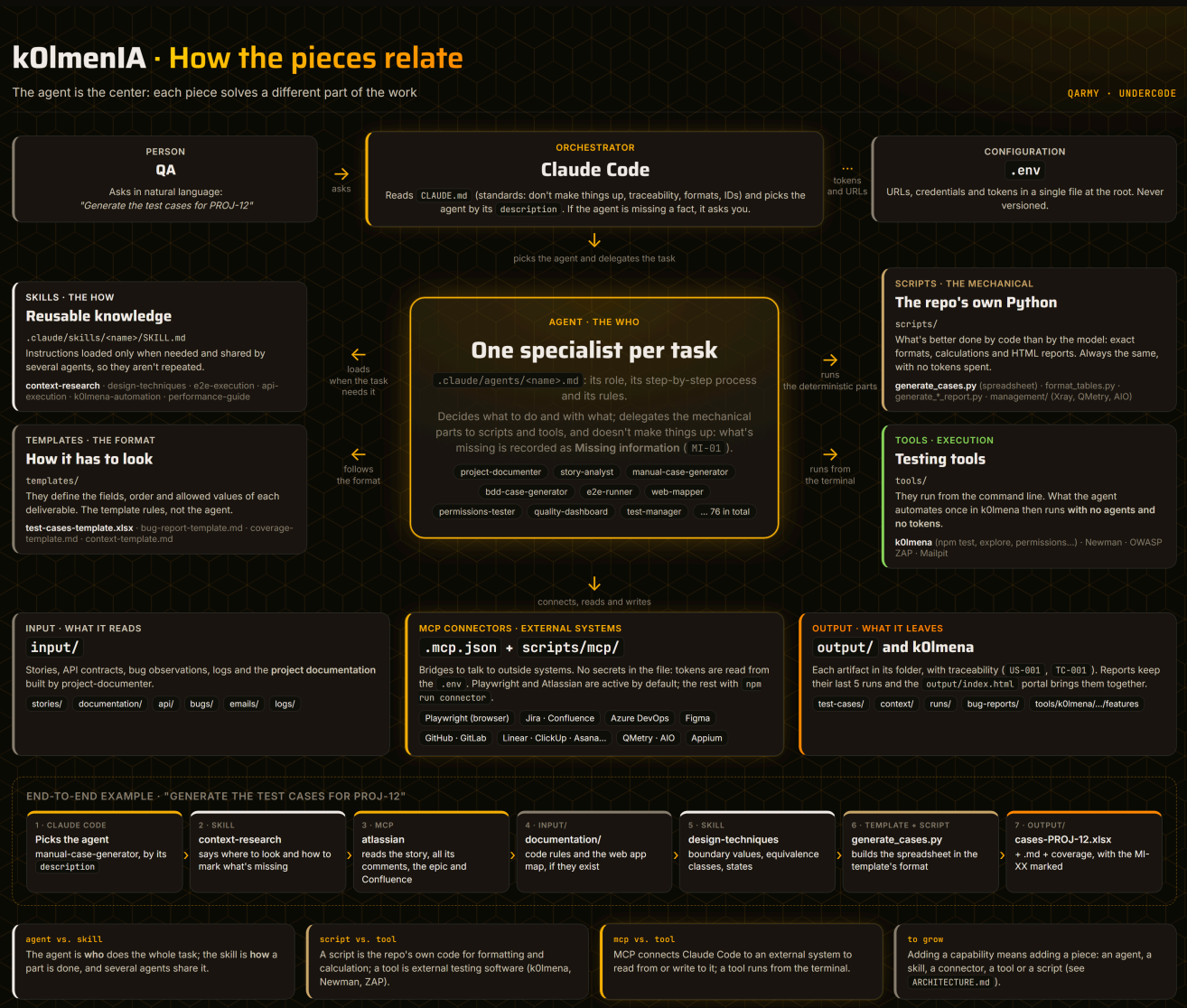
Your tool + `output/`

Principles

principle	in practice
Don't make things up	If a step, a piece of data, a rule or a threshold is missing, the agent flags it and asks. Better an artifact with marked gaps than a complete but invented one.
Respect the formats	Fields, their order and their values are defined by the templates and scripts, not by the agent.
Traceability	Everything references the story (`US-XXX`); every automated scenario carries the test case ID (`@TC-001`) and the key in the test management tool.
Thoughtful coverage	Positive, negative, edge cases and field validations, not just the happy path.
Research before assuming	Before working on a story, the agents review the project documentation (repository, PRs, map of the web app), its comments, subtasks, epic, linked issues, Confluence and Figma. Whatever doesn't show up is marked as **Missing information** (`MI-01`).
Token savings	What repeats is automated once and then runs without agents.
Security	Secrets live in the `.env` at the root, which is never committed; the agents never ask for tokens in the chat.

Architecture





piece	what it is	where
Claude Code	Orchestrator: interprets the request, picks the agent and applies the project standards.	`CLAUDE.md`
Agents	The "who": one specialist per task.	`.claude/agents/`
Skills	The "how": knowledge loaded only when needed (design techniques, E2E and API execution, k0lmena conventions).	`.claude/skills/`
Templates and scripts	Output format (test case spreadsheet, coverage, bug report) and deterministic HTML reports.	`templates/` · `scripts/`
Test management integration	`management.py` with one adapter per tool: Xray Cloud, Xray Server/DC, QTM4J and AIO Tests. Azure DevOps goes through its MCP connector.	`scripts/management/`
MCP connectors	Playwright and Atlassian (Jira and Confluence) active; Azure DevOps, Figma, GitHub, GitLab, Linear, ClickUp, monday.com, Asana, Trello, Notion, YouTrack, Shortcut, Appium, QMetry, AIO Tests and k0lmenaTMT ready to enable. Pinned versions, no secrets in the file.	`.mcp.json`
Tools	Newman for Postman collections and k0lmena for web, API, mobile and performance.	`tools/`

// introduction

How to use it



01 Provide your inputs

Ask *"Document the project"* to gather the context from the repository, the stories and the web app into `input/documentation/` ; put everything else in `input/` , or pass a Jira key, a Confluence page or a Figma link with the connector active. The agents research the full context and flag what's missing (`MI-01`).

02

Ask in natural language

Claude Code picks the agent. You can also name it: *"use the web-mapper to..."*.

03

Review the result

In `output/` or in `tools/k0lmena/`.

04

Automate what repeats

And run it with `npm test` as many times as you want, with no tokens.

#	you ask	you get
0	"Document the project" · "Gather the context of PROJ-12"	Project knowledge base + story dossier in `input/documentation/`
1	"Research the context of PROJ-12"	Context sheet + Missing information
2	"Analyze story US-001"	Ambiguities and questions for the PO
3	"Build the test plan for US-001"	HTML plan
4	"Generate the test cases for US-001"	`cases-US-001.xlsx` + coverage
5	"Upload the test cases to Xray in 'US-001 Signup', linked to PROJ-12, and create the Sprint 5 cycle"	Test cases and cycle in Xray
6	"Automate the US-001 test cases in k0lmena against https://your-app.com"	`.feature` + steps + locators
7	`npm test` · `npm run report:web`	HTML report with evidence
8	"Upload the results to the Sprint 5 cycle"	Statuses and evidence in Xray
9	"Build a load test of the login for 20 users"	k6/Artillery/JMeter script + HTML report
10	"Which test cases should I run for PR #42?"	Risk-based selection + `npm test` command with tags
11	"Run a visual regression of https://your-app.com" · "Test US-003 in Chrome, Firefox and Safari"	What moved and by how many px · per-browser matrix
12	"Repair the tests that failed because of locators"	New locators applied and verified
13	"Check that the welcome email reaches ana@test.com" · "Review the site's Spanish and Portuguese translations"	Emails checked against expectations · percentage translated per language
14	"Is this bug already filed in Jira?" · "What happens if a blocked user tries to pay?"	Duplicates and regressions · answer with its source
15	"Test the admin, operator and guest permissions" · "What does the user see if the API goes down?"	Role × route matrix · page × scenario matrix
16	"Build the closing report for US-001"	Go / no-go
17	"Build the sprint 12 dashboard" · "Post the result to Slack"	Trend and management summary · notification in Slack or Teams

// introduction

Agents

Claude Code picks the agent based on what you ask. They live in `.claude/agents/` and are grouped into four families. The full catalog (when to use each one, example request, inputs, outputs and what it asks you to confirm) is in `AGENTS.md`.

Analysis and design

project-documenter

Builds the project documentation for the other agents: reads the repository (read-only), the stories with their comments, PRs and links, and makes a functional map of the web app.

`skills` context-research, e2e-execution
`uses` npm run functional-map, documentation_index.py
`connectors` github, gitlab, azure-devops, atlassian, playwright

`input/documentation/`

story-analyst

Researches the story context and detects ambiguities, gaps, risks and Missing information.

`skills` context-research
`uses` format_tables.py
`connectors` atlassian, azure-devops, figma

`output/story-analysis/`

test-strategist

Test plan: scope, risks, test types, environment and criteria, in HTML.

`skills` context-research
`uses` generate_plan.py
`connectors` atlassian, azure-devops, figma

`output/test-plans/`

manual-case-generator

Test cases in Excel and Markdown + a coverage report with questions for the PO.

`skills` context-research, design-techniques
`uses` generate_cases.py
`connectors` atlassian, azure-devops, figma

`output/test-cases/manual/`

bdd-case-generator

Gherkin scenarios (in English) + coverage.

`skills` context-research, design-techniques
`uses` format_tables.py
`connectors` atlassian, azure-devops, figma

`output/test-cases/bdd/`

api-case-generator

Positive, negative, schema and authorization API test cases.

`skills` context-research, design-techniques
`uses` format_tables.py
`connectors` atlassian, azure-devops

`output/api-cases/`

test-data-generator

Valid, invalid and edge-case data in Markdown or CSV.

`skills` context-research
`uses` format_tables.py
`connectors` atlassian, azure-devops

`output/test-data/`

test-case-selector

What should I test with this change? It crosses a diff, a PR, issues or release notes with all the test cases and builds a risk-based selection, with the `npm test` command ready.

`skills` context-research
`uses` select_cases.py, generate_selection_report.py
`connectors` atlassian, azure-devops

`output/case-selection/`

case-reviewer

Reviews existing test cases: vague steps or missing expected results, duplicates, stories without negative cases, a score per case and how long they take to run by hand.

`skills` context-research, design-techniques
`uses` review_cases.py, generate_review_report.py, estimation.py
`connectors` atlassian, azure-devops

`output/case-review/`

charter-generator

Session-based exploratory testing: charters with mission, focus, heuristics and time box, and the sessions report built from the notes.

`skills` context-research, design-techniques, e2e-execution
`uses` generate_charters.py
`connectors` atlassian, azure-devops, playwright

`output/charters/`

bug-triager

Before a bug is filed, it looks for duplicates and possible regressions (Jira, Azure DevOps, local bugs, repository issues) and suggests severity, component, owner and story.

`uses` find_duplicates.py, notify.py
`connectors` atlassian, azure-devops, github, gitlab

`output/triage/`

project-consultant

Answers questions about the product using the project documentation, citing the source of every fact or saying it isn't documented.

`skills` context-research
`uses` input/documentation/

`chat · output/consultations/`

onboarding-guide

Onboarding plan for a new QA: the product with its sources, their environment, the agents for their role, the first tasks and a checklist.

`uses` generate_onboarding.py, doctor.py

`output/onboarding/`

project-researcher

Surveys the whole project from a board URL: it drills down epics → stories → tasks → comments → reference links. It delivers a web with concept maps (architecture and how it works), a short summary and a slide deck (prints to PDF), with ambiguities (AMB-XX) and prioritized questions (P-XX) for a new QA.

`skills` project-survey, context-research
`uses` generate_research_report.py
`connectors` atlassian, azure-devops, github, gitlab, figma

`output/research/`

bug-report-generator

Professional bug report following the template, with justified severity and priority.

`uses` bug template, format_tables.py
`connectors` atlassian, azure-devops

`output/bug-reports/`

Live execution `uses tokens on every run`

e2e-runner

Runs test cases in a real browser with Playwright MCP (headed or headless), with evidence.

`skills` e2e-execution
`uses` generate_report.py
`connectors` playwright, playwright-headless

`output/runs/`

api-runner

Runs a Postman, Bruno or Insomnia collection with Newman (REST and GraphQL).

`skills` api-execution
`uses` Newman, run_newman.py

`output/runs/`

web-explorer

Exploratory testing: broken links and images, JavaScript and console errors, and responsive layout at several resolutions, with screenshots and a report.

`skills` e2e-execution
`uses` npm run explore, generate_exploratory_report.py
`connectors` playwright, playwright-headless

`output/exploratory/`

accessibility-analyst

WCAG 2.2 audit with axe-core, keyboard and reflow, plus a manual review: failed criteria, screenshots and how to fix each one.

`skills` e2e-execution
`uses` npm run accessibility, generate_accessibility_report.py
`connectors` playwright, playwright-headless

`output/accessibility/`

web-performance-auditor

Load speed with Lighthouse on mobile and desktop: Core Web Vitals, opportunities with estimated savings and the loading sequence.

`uses` npm run web-performance (Lighthouse), generate_web_performance_report.py

`output/web-performance/`

pixel-perfect

Visual regression against a reference (or the design): what moved and by how many px, what's missing, what changed in text or color, with a slider comparator.

`skills` e2e-execution
`uses` npm run pixel-perfect, generate_pixel_perfect_report.py
`connectors` playwright, figma

`output/pixel-perfect/`

cross-browser

Chromium (Chrome and Edge), Firefox and WebKit (Safari): the k0lmena suite on each one or the pages compared side by side; separates what fails in only one browser.

`skills` e2e-execution
`uses` npm run cross-browser, generate_cross_browser_report.py
`connectors` playwright, playwright-headless

`output/cross-browser/`

email-verifier

The app's emails with a test mailbox (Mailpit): the expected email arrives, no unreplaced variables, working links and images, compatible with Outlook and Gmail, and how it looks on desktop and mobile.

`skills` context-research, e2e-execution, k0lmena-automation
`uses` Mailpit, npm run emails, generate_emails_report.py
`connectors` playwright, playwright-headless

`output/emails/`

translation-analyst

Every page in every language: untranslated text, visible keys, broken characters, cut-off text and number and date formats from another region, with the proposed translation.

`skills` context-research, e2e-execution
`uses` npm run translator, generate_translation_report.py
`connectors` playwright, playwright-headless

`output/translator/`

ux-evaluator

Usability with Nielsen's 10 heuristics: a score per heuristic, automatic findings and findings from the manual walkthrough with screenshots, the 404 page, desktop and mobile.

`skills` e2e-execution, context-research
`uses` npm run ux, generate_ux_report.py
`connectors` playwright, playwright-headless

`output/ux/`

permissions-tester

Logs in with every role and builds the matrix of what each one sees and can open (also by direct URL and in the API), compared with the documented rule.

`skills` context-research, e2e-execution
`uses` npm run permissions, generate_permissions_report.py
`connectors` playwright, playwright-headless

`output/permissions/`

resilience-tester

Simulates a down or slow API, no connection, broken responses and static files that don't load, and checks for blank screens, endless spinners and recovery.

`skills` e2e-execution, context-research
`uses` npm run resilience, generate_resilience_report.py
`connectors` playwright, playwright-headless

`output/resilience/`

form-tester

Validations of every field without submitting the form: required fields, lengths, formats and limits, invalid values accepted and valid ones rejected, with the message and a screenshot.

`skills` e2e-execution, context-research
`uses` npm run forms, generate_forms_report.py
`connectors` playwright, playwright-headless

`output/forms/`

html-report-generator

HTML dashboard of a test run's results.

`uses` generate_report.py

`output/runs/`

closure-report-generator

Closing report of the test round with a go/no-go recommendation.

`uses` collect_results.py, generate_closure_report.py

`output/closure-reports/`

quality-dashboard

Dashboard per sprint or release: pass rate and trend, % automated, coverage, bugs, flaky tests and a management summary.

`uses` quality_dashboard.py, reports_portal.py

`output/dashboard/`

coverage-analyst

Traceability matrix (story → criteria → test cases → automation → results) and what's left to test.

`skills` context-research
`uses` analyze_coverage.py, generate_coverage_report.py
`connectors` atlassian, azure-devops

`output/coverage/`

streaming-tester

Streaming and playback with ffprobe/ffmpeg (RTSP/ONVIF, RTMP, HLS, DASH, WebRTC, VOD): that the source comes up, codec, resolution, FPS, bitrate, audio, keyframes, startup latency, continuity, stability (soak with reconnection) and concurrent load.

`skills` streaming-verification, context-research
`uses` npm run streaming (ffprobe/ffmpeg), generate_streaming_report.py

`output/streaming/`

onvif-tester

ONVIF control of an IP camera: device info, profiles and stream URIs, snapshot, clock/drift, timezone and NTP, firmware vs. the site standard, PTZ (optional) and events/live motion detection (pull-point). Returns the RTSP URI for streaming-tester.

`skills` onvif-control, context-research
`uses` npm run onvif (onvif-zeep), generate_onvif_report.py

`output/streaming/onvif-*/`

image-quality-analyst

Camera image quality: captures a frame and detects black frames, blur and freeze with OpenCV, and with vision a dirty or blocked lens, a moved camera, backlight and night IR; compares against a reference.

`skills` image-quality, context-research
`uses` npm run image-quality (ffmpeg + OpenCV), generate_quality_report.py

`output/streaming/quality-*/`

recording-verifier

Camera or NVR recordings over ONVIF (Recording/Search): what's recorded, the covered range, gaps and whether the expected retention is met. Reports if the device doesn't support it over ONVIF.

`skills` onvif-control, context-research
`uses` npm run recordings (onvif-zeep), generate_recordings_report.py

`output/streaming/recordings-*/`

camera-security-analyst

Passive security analysis of IP cameras/NVR: open ports and insecure services (Telnet, FTP, HTTP without TLS, RTSP without authentication) and firmware to review CVEs. It does not test credentials or attack.

`skills` camera-security, context-research
`uses` npm run camera-security, generate_camera_security_report.py

`output/streaming/camera-security-*/`

salesforce-tester

Tests a Salesforce org (sandbox): read-only survey with the sf CLI (profiles, permissions, OWD/FLS, validation rules, flows, triggers, Apex coverage, limits), derives tests by risk, runs Apex tests and executes per profile in Lightning with Playwright. Never production; it does not touch metadata.

`skills` salesforce-testing, token-testing, e2e-execution, context-research
`uses` npm run salesforce, generate_salesforce_report.py
`connectors` salesforce, playwright

`output/salesforce/`

wallet-tester

Digital wallets and payments in sandbox: balance conservation and ledger, transaction states, idempotency, double-spend and concurrency, limits, KYC/AML, reconciliation with the gateway and security of tokens and cards (PCI). Test money and test cards, never real ones.

`skills` wallet-testing, token-testing, design-techniques, e2e-execution, context-research
`uses` generate_wallets_report.py; api-runner, data-verifier
`connectors` playwright

`output/wallets/`

shopify-tester

Tests a Shopify store: read-only survey of the storefront (published theme, Liquid errors, policies, catalog and cart) and the Admin API (development store, products and stock, markets, shipping, discounts, webhooks), runs Theme Check, derives tests by risk and runs them on mobile and desktop with the Bogus Gateway or test mode. It does not change the store.

`skills` shopify-testing, ecommerce-testing, e2e-execution, context-research
`uses` npm run shopify, generate_ecommerce_report.py
`connectors` playwright

`output/shopify/`

wordpress-tester

Tests WordPress and WooCommerce: read-only survey of the storefront, the WooCommerce REST API (pending database update, cron, outdated plugins and templates, pages, gateways and their mode, shipping, coupons, webhooks) and WP-CLI, derives tests by risk and tests checkout, payments, shipping, coupons, emails and roles. It does not update or change anything.

`skills` wordpress-testing, ecommerce-testing, e2e-execution, context-research
`uses` npm run wordpress, generate_ecommerce_report.py
`connectors` playwright

`output/wordpress/`

prestashop-tester

Tests PrestaShop (1.6/1.7/8.x): read-only survey of the storefront and the Webservice API (products, stock, combinations, carriers, zones, currencies, vouchers, multistore and performance config) and, with --local, version and dev mode; local smoke, checkout, test payments, stock, rounding and performance.

`skills` prestashop-testing, ecommerce-testing, e2e-execution, performance-guide, context-research
`uses` npm run prestashop, generate_ecommerce_report.py
`connectors` playwright

`output/prestashop/`

vtex-tester

Tests VTEX: storefront, public Catalog System (SKUs, price, stock) and, with a read-only app, sales channels, shipping SLAs and promotions; smoke on a QA workspace, checkout/orderForm, sandbox payments, channels, promos and OMS.

`skills` vtex-testing, ecommerce-testing, e2e-execution, performance-guide, context-research
`uses` npm run vtex, generate_ecommerce_report.py
`connectors` playwright

`output/vtex/`

sap-tester

Tests SAP (S/4HANA or ECC) on a QA client: SAP GUI (T-codes via the Scripting API) and Fiori/UI5 (OData), authorizations and roles (PFCG/SU53/SoD) and end-to-end O2C/P2P/R2R processes; smoke, regression after a transport, performance and security.

`skills` sap-testing, e2e-execution, api-execution, performance-guide, context-research
`uses` npm run sap, generate_sap_report.py
`connectors` playwright

`output/sap/`

desktop-tester

Tests Windows desktop apps (WinForms/WPF/UWP/Win32/Electron/Java/Qt) surveys the control tree (pywinauto) and tests with Appium+WinAppDriver, pywinauto or Playwright (Electron); functional, install, files, OS, regression, performance and security.

`skills` desktop-testing, e2e-execution, performance-guide, context-research
`uses` npm run desktop, generate_desktop_report.py
`connectors` appium, playwright

`output/desktop/`

mobile-tester

Tests mobile apps (Android/iOS; native/hybrid/web) live with Appium: permissions, interruptions, offline, deep links and push, lifecycle, OS and screen compatibility, performance and mobile security (MASVS).

`skills` mobile-testing, performance-guide, context-research
`uses` generate_mobile_report.py
`connectors` appium

`output/mobile/`

dependency-analyst

Dependency SCA (read-only): runs npm audit / pip-audit / osv-scanner, aggregates CVEs by severity, tells direct from transitive and vulnerable from outdated, and flags licenses to review.

`skills` dependency-analysis, context-research
`uses` npm run dependencies, generate_dependencies_report.py

`output/dependencies/`

secure-code-analyst

SAST and secrets (read-only): built-in secret scanner (redacts the value) + semgrep if present, false-positive triage and OWASP/CWE mapping; complements dynamic analysis (ZAP).

`skills` secure-code, context-research
`uses` npm run sast, generate_sast_report.py

`output/sast/`

ai-tester

Tests AI/LLM features (chatbot, RAG, classifier, agent): golden set and evals, hallucinations and grounding, prompt security (injection/jailbreak), format, robustness, latency and cost.

`skills` ai-testing, api-execution, e2e-execution, context-research
`uses` generate_ai_report.py
`connectors` playwright

`output/ai/`

messaging-tester

Tests messaging/events (Kafka/RabbitMQ/SQS/MQTT/WebSock delivery, ordering, idempotency, DLQ and retries, schemas, consumer lag and outbox; publishes/consumes with the tool.

`skills` async-testing, performance-guide, context-research
`uses` npm run messaging, generate_messaging_report.py

`output/messaging/`

soap-tester

Tests SOAP services from the WSDL: operations, XSD validation, SOAP Faults, namespaces/SOAPAction, WS-Security and idempotency; sends envelopes with api-runner.

`skills` soap-testing, api-execution, context-research
`uses` npm run soap, generate_soap_report.py

`output/soap/`

contract-tester

Contract testing (Pact): consumer-driven and bi-directional for REST, GraphQL, gRPC and events; provider verification, can-i-deploy and compatibility.

`skills` contract-testing, api-execution, context-research
`uses` generate_generic_report.py

`output/contracts/`

seo-analyst

SEO and GEO: indexability, on-page, structured data, hreflang and AI crawler access; surveys with npm run seo and defers Core Web Vitals to web-performance-auditor.

`skills` seo-testing, context-research
`uses` npm run seo, generate_generic_report.py

`output/seo/`

data-tester

Data and pipelines (ETL): quality by dimension, source-target reconciliation, schema validation, transformations and idempotency; read-only with data-verifier.

`skills` data-testing, context-research
`uses` generate_generic_report.py

`output/data/`

dynamics-tester

Dynamics 365 and other CRM/ERP (sandbox): roles, logic (flows/plugins/BPF/roll-ups) and end-to-end processes; read-only Web API + Playwright per role.

`skills` dynamics-testing, e2e-execution, context-research
`uses` generate_generic_report.py
`connectors` playwright

`output/dynamics/`

blockchain-tester

Blockchain/Web3 and smart contracts (testnet/local): functional, tokens, gas, security (reentrancy, access control, oracles, proxies), Hardhat/Foundry and invariants.

`skills` blockchain-testing, api-execution, context-research
`uses` generate_generic_report.py

`output/blockchain/`

ivr-tester

Voice/IVR and voicebots: menu tree (DTMF/voice), prompts, routing and queues, telephony (SIP/VoIP) and compliance (PCI).

`skills` ivr-testing, context-research
`uses` generate_generic_report.py

`output/ivr/`

game-tester

Video games (Unity/Unreal/Godot/HTML5/mobile): mechanics, saves, compatibility, performance, multiplayer, monetization and world-breaking exploratory testing.

`skills` game-testing, context-research
`uses` generate_generic_report.py

`output/games/`

pos-tester

Point of sale (POS) and hardware: sale and cash count, payments and peripherals, fiscal, offline and reconciliation, cash close.

`skills` pos-testing, context-research
`uses` generate_generic_report.py

`output/pos/`

observability-analyst

Analyzes observability: validates logs (trace id, no PII), metrics (Prometheus RED/USE), traces (OTel), healthchecks (liveness/readiness), error tracking, SLOs and that alerts fire; read-only survey of health and /metrics.

`skills` observability-testing, api-execution, context-research
`uses` npm run observability, generate_generic_report.py

`output/observability/`

search-tester

Tests search and relevance (Elasticsearch/Algolia/Solr/DB): ranking and boosting, typos and synonyms, facets, zero-results, indexing and security, with a golden set and metrics (precision@k, nDCG).

`skills` search-testing, api-execution, e2e-execution, context-research
`uses` generate_generic_report.py
`connectors` playwright

`output/search/`

document-tester

Tests generated documents (PDF/Excel/CSV): correct data and totals, unreplaced placeholders, format/locale, layout, metadata and security; extracts content with pypdf/openpyxl.

`skills` document-testing, context-research
`uses` npm run documents, generate_generic_report.py

`output/documents/`

content-reviewer

Reviews content and typos in the main language: spelling, visible placeholders and i18n keys, duplicate words, Lorem ipsum and microcopy consistency (distinct from translation-analyst).

`skills` content-testing, e2e-execution, context-research
`uses` npm run content, generate_generic_report.py
`connectors` playwright

`output/content/`

mutation-analyst

Mutation testing: measures the real quality of the test suite (Stryker/mutmut/PITest), computes the mutation score per file, tells surviving mutants from equivalent ones and points out the missing test. A better signal than line coverage.

`skills` mutation-testing, context-research
`uses` npm run mutation, generate_generic_report.py

`output/mutation/`

Automation with k0lmena

`then runs with no tokens`

web-mapper

Walks the web app following your test cases and generates the `.feature` files, steps and locators.

`skills` k0lmena-automation, e2e-execution, context-research
`uses` k0lmena (Playwright + Cucumber)
`connectors` playwright, playwright-headless

`k0lmena/web/`

api-mapper

Reads Postman or Swagger/OpenAPI, verifies the endpoints and generates the `.feature` files.

`skills` k0lmena-automation, context-research, design-techniques
`uses` k0lmena (axios + Cucumber)

`k0lmena/api/`

mobile-mapper

Walks the app with Appium and generates the `.feature` files, steps and locators.

`skills` k0lmena-automation, context-research
`uses` k0lmena (WebdriverIO + Appium)
`connectors` appium-mcp

`k0lmena/mobile/`

performance-mapper

Load, stress, soak and spike tests with k6, Artillery or JMeter; asks for the load and thresholds.

`skills` performance-guide, k0lmena-automation
`uses` k6, Artillery, JMeter, generate_performance_report.py

`k0lmena/performance/`

data-verifier

Checks data in PostgreSQL, MySQL, SQL Server or MongoDB and adds database checks to the tests.

`skills` k0lmena-automation
`uses` npm run db

Chat · steps in `.feature`

failure-analyst

When `npm test` fails: separates real bugs from broken, flaky, data or environment tests, with the evidence.

`skills` e2e-execution
`uses` analyze_failures.py, generate_failures_report.py

`output/failures/`

log-analyst

Server logs (app, nginx, Apache, JSON, syslog) grouped by signature with their stack trace and crossed with the run: which server error happened while each scenario failed.

`uses` analyze_logs.py, generate_logs_report.py

`output/logs/`

automation-repairer

When a test fails because of a locator or a wait: diagnoses it with the DOM saved at failure time, proposes the new locator, applies it with a backup and verifies it. If it's a bug, it doesn't touch the test.

`skills` k0lmena-automation, e2e-execution
`uses` npm run repair, generate_repair_report.py
`connectors` playwright, playwright-headless

`output/repairs/`

security-analyst

Passive web security scan with OWASP ZAP and a report with findings, evidence and how to fix them.

`uses` OWASP ZAP (Docker), generate_security_report.py

`output/security/`

web-pentester

Authorized penetration test (ethical hacking): active OWASP ZAP scan plus manual OWASP Top 10 verification with Playwright; confirms each finding and builds the report with steps to reproduce and how to fix. Authorized URLs only, with permission.

`skills` e2e-execution, context-research
`uses` active OWASP ZAP (Docker), generate_security_report.py
`connectors` playwright, playwright-headless

`output/security/`

Test management

test-manager

Folders, test cases, links to the story and cycles in Xray, QMetry (QTM4J), AIO Tests or Azure DevOps Test Plans.

`uses` scripts/management/management.py
`connectors` azure-devops (Azure Test Plans)

The tool + `output/management/`

results-publisher

Uploads the `npm test` results to the cycle: status, error and evidence.

`uses` scripts/management/management.py

The tool

Skills: the "how" the agents use

skill	what it teaches	used by
`context-research`	Research the whole story (project documentation, comments, subtasks, epic, Confluence, Figma, contracts) and record the Missing information (`MI-XX`)	story-analyst, test-strategist, the test case and data generators, web-mapper, api-mapper, mobile-mapper, coverage-analyst, test-case-selector, email-verifier, translation-analyst, case-reviewer, charter-generator, ux-evaluator, form-tester, project-documenter, permissions-tester, resilience-tester, project-consultant
`design-techniques`	Equivalence partitioning, boundary values, decision tables, state transitions and pairwise	the manual, BDD and API test case generators; api-mapper, case-reviewer, charter-generator
`e2e-execution`	Running in the browser with Playwright MCP: modes, snapshots, waits and evidence	e2e-runner, web-mapper, web-explorer, accessibility-analyst, failure-analyst, pixel-perfect, cross-browser, automation-repairer, email-verifier, translation-analyst, charter-generator, ux-evaluator, form-tester, project-documenter, permissions-tester, resilience-tester
`api-execution`	Running collections with Newman: base_url, token, assertions and results	api-runner
`k0lmena-automation`	k0lmena conventions: features, steps, locators, tags, database steps and performance scripts	the mappers, data-verifier, automation-repairer and email-verifier

`performance-guide`	Plan a performance test step by step with the person	the main conversation and performance-mapper
`testing-guide`	Interactive assistant to choose what to test and with which agent/skill: asks step by step and recommends or runs; platform and test-type map	the main conversation
`streaming-verification`	Verify streaming and playback with ffprobe/ffmpeg: protocols, properties, continuity, stability and load	streaming-tester
`onvif-control`	ONVIF camera control: services, profiles, snapshot, safe PTZ, clock/NTP, events and recordings	onvif-tester, recording-verifier
`image-quality`	Assess image quality: OpenCV (black, blur, freeze) and judgment with vision	image-quality-analyst
`camera-security`	Passive security analysis of IP cameras: ports, insecure services and firmware/CVEs	camera-security-analyst
`project-survey`	Survey a whole project over MCP: parse the board URL, drill down epics → stories → tasks → comments → links, sample, build concept maps and questions	project-researcher
`salesforce-testing`	Test a Salesforce org: environments, access (profiles, OWD, FLS), validation rules, flows, triggers, Apex, data, Lightning and releases	salesforce-tester
`token-testing`	Test tokens (JWT, OAuth, session, API keys) and card tokenization (PCI): lifecycle, signature, claims, typical attacks and safe handling	wallet-tester, salesforce-tester, web-pentester
`wallet-testing`	Test wallets and payments: balance and ledger, transactions, idempotency, double-spend, limits, KYC/AML and reconciliation	wallet-tester
`ecommerce-testing`	Test any online store: catalog, prices, stock, cart, checkout, taxes, shipping, coupons, test payments and emails	shopify-tester, wordpress-tester
`shopify-testing`	Test Shopify: development store, themes and Theme Check, Liquid, apps, checkout and Bogus Gateway, Markets, shipping, discounts and webhooks	shopify-tester
`wordpress-testing`	Test WordPress and WooCommerce: plugins, overridden templates, cron, classic or block checkout, gateways, shipping, coupons and roles	wordpress-tester
`prestashop-testing`	Test PrestaShop (1.6/1.7/8.x): local smoke with Docker, themes and Smarty, modules and overrides, cache and performance, checkout, payments, combinations and stock, Webservice API	prestashop-tester
`vtex-testing`	Test VTEX: accounts and workspaces, VTEX IO, catalog and trade policies, pricing, logistics/SLAs, promotions, payments, orderForm, OMS and marketplace	vtex-tester
`sap-testing`	Test SAP: SAP GUI (T-codes, Scripting API) and Fiori/UI5 (OData), authorizations and roles (PFCG/SU53/SoD), O2C/P2P/R2R processes, jobs and IDocs	sap-tester

`desktop-testing`	Test Windows desktop apps (WinForms/WPF/UWP/Electron/Java/Qt): control tree, functional, install, OS, performance, security and accessibility	desktop-tester
`mobile-testing`	Test mobile apps (Android/iOS): devices, permissions, interruptions, connectivity, deep links and push, lifecycle, performance, security (MASVS) and accessibility	mobile-tester
`dependency-analysis`	Analyze dependency security (SCA): CVE/CVSS, direct vs. transitive, licenses, SBOM, supply chain and accepted risk	dependency-analyst
`secure-code`	SAST and secrets: bug classes (injection, XSS, crypto), OWASP/CWE, semgrep, false-positive triage, secret detection and rotation	secure-code-analyst
`ai-testing`	Test AI/LLM features: golden set and evals, hallucinations and grounding, prompt security (injection/jailbreak), format, robustness, latency/cost	ai-tester
`async-testing`	Test messaging/events (async): Kafka/RabbitMQ/SQS/MQTT/WebSocket/SSE, delivery, ordering, idempotency, DLQ, schemas, consumer lag and outbox	messaging-tester
`graphql-testing`	Test GraphQL APIs: schema/introspection, partial errors and nullability, per-field authorization, N+1, depth/complexity and pagination	api-runner, api-mapper, api-case-generator
`grpc-testing`	gRPC (Protobuf, RPC modes, status codes, deadlines, compatibility) and WebSocket in the API family	api-runner, api-mapper, api-case-generator
`soap-testing`	Test SOAP services: WSDL, SOAP 1.1/1.2, XSD validation, SOAP Faults, WS-Security, mustUnderstand and contract	soap-tester
`contract-testing`	Contract testing (Pact): consumer-driven and provider, broker/can-i-deploy, compatibility, REST/GraphQL/gRPC/events	contract-tester
`seo-testing`	SEO and GEO: indexability, on-page, structured data, hreflang, Core Web Vitals and optimization for AI engines	seo-analyst
`data-testing`	Data and pipelines (ETL): quality, reconciliation, schema/drift, transformations, SCD, idempotency and PII	data-tester
`dynamics-testing`	Dynamics 365 and other CRM/ERP: environments, roles, data model, logic, end-to-end processes and Web API	dynamics-tester
`blockchain-testing`	Blockchain/Web3 and smart contracts: functional, tokens, gas, security (reentrancy, access, oracles) and proxies	blockchain-tester
`ivr-testing`	Voice/IVR and voicebots: menu tree, DTMF/ASR, prompts, queues and transfers, telephony and PCI	ivr-tester

`game-testing`	Video games: mechanics, saves, compatibility, performance, multiplayer, monetization and exploratory	game-tester
`pos-testing`	POS and hardware: sale flow, payments and peripherals, fiscal, offline and reconciliation, cash close	pos-tester
`observability-testing`	Test observability: logs (trace id, no PII), Prometheus metrics (RED/USE, cardinality), OTel traces, healthchecks, SLOs and alerts	observability-analyst
`search-testing`	Test search and relevance: ranking, typos/fuzzy, synonyms, facets, zero-results, indexing and golden set (precision@k, nDCG)	search-tester
`document-testing`	Test generated documents (PDF/Excel/CSV): data and calculations, placeholders, format/locale, layout, metadata, security and PDF a11y	document-tester
`content-testing`	Review content and typos: spelling/grammar, placeholders, visible i18n keys, duplicate words, microcopy and consistency	content-reviewer
`mutation-testing`	Mutation testing: mutants and states (killed/survived/no-coverage), mutation score, survivors vs. equivalents, bounded cost (diff/module) and per-language tools	mutation-analyst
`accessibility-testing`	Accessibility (a11y) on web (WCAG 2.2, axe-core), mobile (TalkBack/VoiceOver, labels) and desktop (UI Automation, accessible names)	accessibility-analyst, mobile-tester, desktop-tester

// getting started

Requirements

Install only what you'll use.

to…	you need
Use the agents	**Claude Code** and a Claude account (Pro, Max, Team or Enterprise) or Anthropic API access. VS Code recommended.
Test cases, reports and management	**Python 3** with `openpyxl`, `tabulate` and `requests`.
Run live E2E	**Node.js 18+** and the Playwright browsers.
Run Postman collections	**Newman**.
Automate with k0lmena	**Node.js 20+**.
Automate mobile	**Node.js 22+**, JDK and Android SDK, or macOS with Xcode for iOS; or a BrowserStack account.

// getting started

Installation

In five minutes you have the agents running; k0lmena and the rest can be added when you need

k0lmenIA · Getting started

Install, set your tokens in the `.env` and start asking the agents for things

QARMY · UNDERC0DE

1 Get it running

A few minutes, only once

1 Requirements

Python 3.10+ · Node.js 20+ (22 LTS recommended) · VS Code + Claude Code (Claude account or Anthropic API). Optional: Java 8+ (JMeter), Docker (security), JDK + Android SDK or Xcode (mobile).

2 Clone the repository

```
git clone
https://github.com/underc0delabs/k0lmenIA.git
cd k0lmenIA
```

3 Install the dependencies

```
pip install -r requirements.txt
# Mac/Linux: pip3

npm install
# installs k0lmena
```

4 Create your `.env`

```
cp .env.example .env
# PowerShell: copy
```

Fill in only what you'll use (column 2).

5 Check that everything is ready

```
npm run doctor
# says what's missing and how
```

6 Open Claude Code

```
code .
claude
# asks you to sign in the 1st time
```

For extra browsers: `cd tools/k0lmena` · `npx playwright install chromium firefox webkit`

2 Set tokens in the `.env`

A single file, at the root · never versioned · the agents never ask for tokens in the chat

App under test — E2E, mappers, exploratory, visual

```
APP_URL=https://staging.your-app.com
APP_USER=… · APP_PASSWORD=…
BASEURL=(if empty, uses APP_URL)
```

API — Newman, API suite, k6

```
API_BASEURL=https://api.your-app.com · API_TOKEN=<your-token>
```

Jira and Confluence — atlassian connector (active)

```
OAuth from /mcp, or with a token:
ATLASSIAN_MCP_AUTHENTICATION=token
JIRA_BASE_URL · JIRA_EMAIL · JIRA_API_TOKEN
```

Test management — scripts/management

```
MANAGEMENT_TOOL=xray-cloud | xray-dc | qtm4j | aio
MANAGEMENT_PROJECT=PROJ
Xray Cloud: XRAY_CLOUD_CLIENT_ID · XRAY_CLOUD_CLIENT_SECRET
Xray DC: XRAY_DC_URL · XRAY_DC_TOKEN
QMetry: QTM4J_API_KEY · AIO: AIO_API_TOKEN
Optional: ESTIMATION_FACTOR=1.3 (estimated time × 1.3)
```

App repository — project-documenter

```
REPO_PROVIDER=github | gitlab | azure-repos | bitbucket
REPO_NAME=my-company/my-app · GITHUB_TOKEN · GITLAB_TOKEN
```

Azure DevOps — azure-devops connector

```
ADO_ORGANIZATION · ADO_PAT · ADO_PROJECT
```

Databases and security

```
DB_CONNECTIONS=main + DB_MAIN_ENGINE · _HOST · _DATABASE · _USER · _PASSWORD
SECURITY_AUTHORIZED_URLS=https://staging.your-app.com
```

Email, languages, roles and notifications — email, translator, permissions, Slack/Teams

```
MAILPIT_URL=http://localhost:8025
TRANSLATOR_LANGUAGES=en-US,es-AR,pt-BR
PERMISSIONS_ROLES=admin,operator + PERMISSIONS_<ROLE>_USER · _PASSWORD
SLACK_WEBHOOK_URL · TEAMS_WEBHOOK_URL · MONITORING_URLS
Mailpit: docker compose -f tools/mailpit/docker-compose.yml up -d
```

Optional connectors (Azure DevOps, Figma, GitHub, GitLab, Linear, ClickUp, monday, Asana, Trello, Notion, YouTrack, Shortcut, QMetry, AIO Tests, k0lmenaTMT, Appium): `npm run connector -- activate <name>`, restart Claude Code and check with `/mcp`. Use test credentials, never production ones.

3 Ask the agents

In natural language: Claude Code picks the agent (or you name it)

ANALYSIS AND DESIGN

- *"Document the project"* — `project-documenter` NEW repository, stories, links and web app
- *"Analyze story US-001 and tell me what's left to define"* — `story-analyst`
- *"Generate the test cases for PROJ-12"* — `manual-case-generator` Excel with estimated time + coverage
- *"Review the US-001 test cases and tell me how long they take"* — `case-reviewer` score, duplicates and minutes
- *"Which test cases should I run for PR #42?"* — `test-case-selector` risk-based selection + command
- *"Is this bug already filed in Jira?"* — `bug-triager` NEW duplicates and regressions
- *"What happens if a blocked user tries to pay?"* — `project-consultant` NEW answers with the source
- *"Build the onboarding for Ana, our new manual QA"* — `onboarding-guide` NEW plan and checklist

LIVE EXECUTION

- *"Run a visual regression of qarmy.ar"* — `pixel-perfect` what shifted and by how many px
- *"Evaluate the checkout's usability"* — `ux-evaluator` Nielsen's heuristics
- *"Test the sign-up form's validations"* — `form-tester` field by field, without submitting
- *"Test the admin, operator and guest permissions"* — `permissions-tester` NEW role × route matrix
- *"What does the user see if the API goes down?"* — `resilience-tester` NEW simulated failures
- *"Check that the welcome email reaches ana@test.com"* — `email-verifier` with Mailpit
- *"Run exploratory tests on the site in the .env"* — `web-explorer` · also accessibility and speed

AUTOMATION WITH K0LMENA

- *"Automate the US-001 test cases in k0lmena"* — `web-mapper` then `npm test`, no tokens
- *"Repair the tests that failed because of locators"* — `automation-repairer` proposes, applies and verifies

MANAGEMENT AND CLOSING

- *"Upload the US-001 test cases to Xray and create the Sprint 5 cycle"* — `test-manager`
- *"Build the closing report for US-001"* — `closure-report-generator` go / no-go
- *"Build the sprint 12 quality dashboard"* — `quality-dashboard` NEW management summary

if a fact is missing
The agent doesn't make it up: it returns **"I need you to confirm"** and Claude asks you.

actions with side effects
Writing to a database, creating data in Jira/Xray or running load: always with your ok.

saves tokens
What's automated runs with `npm test`, with no agents.

everything ends up in
`output/` (HTML reports, with the last 5 runs in `history/`) and `tools/k0lmena/` (automation).

01 Install Claude Code

```
curl -fsSL https://claude.ai/install.sh | bash     # macOS / Linux
npm install -g @anthropic-ai/claude-code           # alternative (Node.js 22 LTS covers everything)
claude --version
```

On Windows, follow the official guide.

02 Clone the repository

```
git clone https://github.com/underc0delabs/k0lmenIA.git
cd k0lmenIA                                        # or the folder you cloned it into
pip install -r requirements.txt                    # Mac/Linux: pip3
npm install                                        # k0lmena
cp .env.example .env                               # and fill it in (PowerShell: copy)
```

On Windows the command is `python`; on Mac and Linux, `python3` and `pip3` (if `pip3` answers *externally-managed-environment*, use a virtual environment: `python3 -m venv .venv`).

03 Check that everything is ready

```
npm run doctor                                     # tells you what's missing and how to fix it
npm run doctor -- --section env,integrations       # only one part · --quick · --json
```

04 Open VS Code and launch Claude Code

```
code .
claude                                             # asks you to sign in the first time
```

05 Add the optional pieces

```
npx playwright install                          # live E2E execution
npm install -g newman                           # Postman collections
cd tools/k0lmena
npx playwright install chromium              # + firefox webkit for cross-browser
npm run bootstrap:k6                            # API performance with k6
npm run bootstrap:jmeter                        # performance with JMeter (Java 8+)
```

// getting started

Configuration: the `.env` file

There is **a single `.env`, at the root of the repo**. The agents, k0lmena and the management scripts use it. It's in `.gitignore` and the commented template is `.env.example`.

group	variables	what for
App under test	`APP_URL` `APP_USER` `APP_PASSWORD`	E2E execution and logins
API	`API_TOKEN` `API_BASEURL` `GRAPHQL_URL`	Newman, the API suite and k6
Web	`BASEURL` `BROWSER` `HEADLESS` `VIEWPORT_WIDTH` `VIEWPORT_HEIGHT` `LOCALE` `TIMEZONE`	Browser for the web suite (empty: Chromium, Firefox and WebKit at 1366×768)
Execution	`TAGS` `PARALLEL`	Which scenarios run and how
Evidence	`EVIDENCE` (`screenshot` · `both` · `off`) `VIDEO` `TRACE`	What gets attached to the report
Auto-healing	`K0LMENA_AUTO_HEALING`	Recover broken locators
Mobile	`MOBILE_TARGET` `MOBILE_PLATFORM` `MOBILE_DEVICE_NAME` `MOBILE_APP` `MOBILE_UDID` `BROWSERSTACK_*`	Where the mobile suite runs
Performance	`PERF_VUS` `PERF_DURATION`	Override the load of the k6 scripts
Databases	`DB_CONNECTIONS` + `DB_<NAME>_ENGINE` `_HOST` `_PORT` `_DATABASE` `_USER` `_PASSWORD` (or `_URL`) `_WRITE` `_PRODUCTION`	Database checks
Test management	`MANAGEMENT_TOOL` `MANAGEMENT_PROJECT` + the tool's credentials	Xray, QMetry and AIO Tests
Azure DevOps (MCP)	`ADO_ORGANIZATION` `ADO_PAT` `ADO_PROJECT`	Boards, Wiki, Repos and Test Plans
Role permissions	`PERMISSIONS_ROLES` `PERMISSIONS_<ROLE>_USER` `_PASSWORD` `_TOKEN` `PERMISSIONS_LOGIN_URL`	Roles tested by permissions-tester

Notifications	`SLACK_WEBHOOK_URL` `TEAMS_WEBHOOK_URL`	Notifications to Slack and Teams (scripts/notify.py)
Monitoring	`MONITORING_URLS` `MONITORING_LINK`	Synthetic monitoring (npm run monitor)
App repository	`REPO_PROVIDER` `REPO_NAME` `REPO_BRANCH` `REPO_LOCAL_PATH` `GITHUB_TOKEN` `GITLAB_URL` `GITLAB_TOKEN`	Read by project-documenter (GitHub, GitLab, Azure Repos or Bitbucket, read-only)
Email	`MAILPIT_URL` `MAILPIT_USER` `MAILPIT_PASSWORD` `EMAIL_TIMEOUT_S`	Mailpit test mailbox (email-verifier and email steps)
Languages	`TRANSLATOR_LANGUAGES` `TRANSLATOR_LANGUAGE_URL`	Languages to review and how the language is switched (translation-analyst)
Estimated time	`ESTIMATION_FACTOR` `MANAGEMENT_NATIVE_TIME`	Optional: multiply the estimated time of the test cases (for example 1.3) and don't use the test management tool's time field (`0`)



Use credentials from a test environment, never from production. The agents read secrets from the `.env` : never paste a token in the chat or write it in a versioned file.

Input folders

folder	what goes there
`input/stories/`	User stories with their acceptance criteria (`US-001.md` , ...).
`input/documentation/`	Project documentation: what project-documenter builds (index, project base, dossier per story) and whatever you add by hand. Not versioned.
`input/api/`	Contracts, Postman collections and environments.
`input/bugs/`	Loose bug observations to turn into reports.
`input/emails/`	What should arrive by email in each case (`expect-US-001.json` , for email-verifier).
`input/logs/`	Application logs for log-analyst (`app.log` , `access.log`). Not versioned.

// guides

Project documentation

project-documenter gathers the real project context and leaves it in `input/documentation/`, so the analyst, the strategist and the test case generators work on what exists and not on assumptions. It's **read-only**: it doesn't write to the repository, to Jira or to the application.

What it reads

source	what it extracts
App repository	README and docs; validations and limits, error messages, states and transitions, roles and permissions, routes and screens, API contracts; and the PRs and commits of each story, with their review comments.
Stories	Jira or Azure DevOps with **all** their comments, subtasks, epic, sibling stories, linked issues and bugs (skill `context-research`).
Links	Confluence, the Azure DevOps Wiki and Figma through their connector; public pages with a quick read. Private ones without a connector (Google Docs, SharePoint, Notion) are listed as **Not accessible**, with what it would take to read them.
The application	`npm run functional-map`: every page with its sections, menus, forms (each field with its label, type, required, lengths, minimums, maximums, pattern and options), buttons, visible messages and tables. GET only, no login and nothing submitted.

What it leaves

file	contents
`INDEX.md`	What documentation exists, which sources it comes from, when it was updated and whether it should be reviewed (`scripts/documentation_index.py`).
`project/`	Product summary (modules, roles, glossary), repository, business rules, messages, states and permissions, functional map and links.
`api/`	The API contracts found in the repository and their summary.
`stories/<US or key>/dossier.md`	Everything about a story: criteria, decisions in comments, links, PRs and touched code, screens, API, **contradictions** between sources and hints for testing.

Every fact carries its source (`src/payments/validations.ts:42`, the PR, the comment with author and date). What comes from the code or the application is recorded as **"how it works today"**, not as a requirement: if it contradicts the story, the `context-research` skill records it as `MI-XX`. That skill reads the index and the dossier first, so every agent that researches a story benefits from it. `input/documentation/` is not versioned.

Connecting the repository

provider	connector	what to configure
GitHub	`github` (remote, read-only)	A read-only fine-grained `GITHUB_TOKEN`. Doesn't support GitHub Enterprise Server.

GitLab	`gitlab` (`/api/v4/mcp`)	`GITLAB_URL` ; your account (OAuth) or a fine-grained `GITLAB_TOKEN` . GitLab.com or 19.5+.
Azure Repos	`azure-devops`	The `repositories` domain is on by default; the PAT needs *Code (Read)*.
Bitbucket Cloud	`atlassian`	`ATLASSIAN_MCP_AUTHENTICATION=token` and an API token with Bitbucket permissions.

In the `.env` : `REPO_PROVIDER` , `REPO_NAME` (one or several, comma-separated), `REPO_BRANCH` and, optionally, `REPO_LOCAL_PATH` (a local clone: the code is read from there, faster and without spending connector tokens). Then run `npm run connector -- activate github` (or `gitlab`) and restart Claude Code.

> *Document the project.*

> *Gather all the context of PROJ-12 before writing the test cases.*

// guides

Context research and "Missing information"

Before analyzing, planning, writing test cases or automating, the agents **research the full context** of the story instead of assuming. Whatever they can't find in any source is marked as **Missing information** in every report.

Where they look

source	what they review
Project documentation	**First.** The index and the story dossier built by project-documenter in `input/documentation/` : if it's up to date, Jira isn't walked again.
Jira story	Description, criteria, status, labels, components and attachments.
Comments	**All of them.** Decisions made later ("we agreed the limit is 50") replace the description, citing who and when.
Subtasks, epic and linked issues	Details of each subtask, general rules and sibling stories in the epic, dependencies and known bugs in the flow.
Confluence	Linked pages or, if there are none, the most relevant ones in the project space.
Figma	Exact texts, error messages, states and required fields.
Contracts and `input/`	Swagger, Postman and local documentation.

The result is a reusable **context sheet**, `output/context/context-US-XXX.md` : sources consulted, findings per criterion, decisions, contradictions and what's missing. If the story didn't change, the other agents reuse it without researching again. If a connector isn't active, the source is recorded as "Not available" instead of being assumed.

> *Research the context of PROJ-12 and generate the test cases.*

Missing information (MI-XX)

Every fact that doesn't appear in any source (or appears contradictory) gets an ID per story, with what's missing, where it was searched, what it affects and the question for the PO. The same ID travels to every artifact. The app isn't a source of requirements: if the expected result depends on a missing fact, the scenario isn't "adjusted" to what the application does; it's automated with the assumption marked or stays `@blocked` until the PO answers.

where	how it looks
Test case spreadsheet	Cases highlighted in amber, tag `@missing-info`, details in Comments and a **Missing information** sheet.
Analyses and coverage	**Missing information** section; criteria with coverage `Partial (missing information)`.
k0lmena `.feature`	Tags `@missing-info @MI-01` and the comment `# MISSING INFORMATION (MI-01): …`.
Mapping report	Status `Missing information`. The mapper doesn't use the app to fill in the fact: what the app shows is recorded as an observation.
k0lmena and execution reports	Visible note on the scenario and a warning in the summary.
Closing report	**Missing information** section; open items count as risk for the go/no-go.
Xray · QMetry · AIO	The result comment clarifies that the test case depends on missing information.

// guides

Design and execution

What to ask, what each agent does and what it leaves.

Analyze a story `story-analyst`

> *Analyze story US-001 and tell me what ambiguities it has.*

It reviews each criterion and delivers `analysis-US-001.md` with ambiguities, gaps, risks, uncovered cases and questions for the PO.

Build the test plan `test-strategist`

> *Build the test plan for US-001.*

HTML dashboard with scope, objectives, test types, risks, data and environment, and entry and exit criteria.

Generate manual test cases `manual-case-generator`

> *Generate the test cases for US-001.*

Delivers `cases-US-001.xlsx`, `cases-US-001.md` and `cases-US-001-coverage.md`. Covers the happy path, negatives, edges and validations. Each case carries its **estimated manual execution time**, with the total at the bottom (see Test case review and estimated time). That spreadsheet is what later gets uploaded to the test management tool or automated.

Generate BDD scenarios `bdd-case-generator`

> *Turn story US-001 into BDD scenarios.*

Generates `US-001-<slug>.feature`, with one `@TC-XXX` tag per scenario, and `US-001-coverage.md`, with the estimated time if they're run by hand.

Generate API test cases `api-case-generator`

> *Generate the authentication API test cases from input/api/auth-endpoints.md.*

Summary table and details per case: method, endpoint, headers, body, expected status and schema.

Test data and bugs `test-data-generator` `bug-report-generator`

> *Generate test data for the sign-up form, in CSV.*

Valid, invalid and edge-case data. Bugs follow `templates/bug-report-template.md` (edit it to use your format) and mark what's missing instead of making it up.

Run live `e2e-runner` `api-runner` `uses tokens`

> *Run ONLY the valid sign-up scenario of US-001 against https://your-app.com.*

The E2E runner asks whether you want to see the browser, runs with Playwright MCP, captures evidence and generates `report-US-001-<date-time>.html`. To walk a whole site looking for failures there's **web-explorer** (see Exploratory testing). The API runner runs the collection with Newman. Ideal for one-off tests: what repeats is better automated.

Reports and closing report `html-report-generator` `closure-report-generator`

> *Build the closing report for the US-001 tests.*

Dashboards with indicators, charts and details; the closing report adds open bugs, risks and the go/no-go recommendation.

// guides

Automation with k0lmena

The framework built into `tools/k0lmena/`. A *mapper* agent walks the application **only once** and writes the automation; from then on the suite runs with `npm`, with no agents and no tokens.

Web

Playwright + Cucumber, video, trace and GIF. With `K0LMENA_AUTO_HEALING=on` it recovers broken locators.

API

axios + Cucumber with generic steps in plain English: no programming needed.

Mobile

WebdriverIO + Appium on a device, emulator or BrowserStack.

Performance

k6 for APIs, Artillery + Playwright for web flows and JMeter for those who already work with .jmx.

The k0lmena folders ship empty: everything inside them is generated by the mappers for your application.

web-mapper

> *Automate in k0lmena the test cases in cases-US-001.xlsx against https://your-app.com.*

1. Reads the test cases (manual or BDD).
2. Navigates the real web app with Playwright MCP and checks that each step can be executed.
3. Generates `web/features/US-001—<slug>.feature`, `web/steps/US-001.steps.ts` and `web/locators/US-001.locators.ts`, reusing existing steps.
4. Validates with `npm run test:web` and leaves `output/mappings/mapping-US-001-web.md`.

If a step can't be executed, the scenario is marked `@blocked` with the reason and doesn't run until it's resolved.

api-mapper

> *Move the /pet endpoints from the Swagger to k0lmena.*

Reads Postman (v2.x) or Swagger/OpenAPI, lets you choose the endpoints, designs positive, negative and authorization scenarios, verifies them against the real API and writes the `.feature` files with the generic steps. Before a POST, PUT, PATCH or DELETE it asks for authorization.

mobile-mapper

> *Automate the app's login test cases in k0lmena on the emulator.*

Needs the Appium MCP connector and the app in `mobile/apps/` (not versioned). After that the suite runs without MCP according to `MOBILE_TARGET`:

target	what you need
`device`	A USB device with debugging enabled and `MOBILE_UDID` (from `adb devices`).
`emulator`	A running emulator or simulator and `MOBILE_DEVICE_NAME`.
`browserstack`	`BROWSERSTACK_USER`, `BROWSERSTACK_KEY` and `BROWSERSTACK_APP`.

Running the suite

```
cd tools/k0lmena
npm test                                  # web + API
npm run test:web                          # also: test:api · test:mobile · test:all
TAGS=@US-001 npm test                     # filter by tag (bash)
$env:TAGS="@US-001"; npm test             # filter by tag (PowerShell)
npm run report:web                        # also: report:api · report:mobile
```

If `EVIDENCE` is empty, `npm test` asks at startup what to keep from passing tests: just a screenshot, or a screenshot + a GIF of the run.

Evidence in the report

suite	passing scenario	failing scenario
Web	Final screenshot (+ optional GIF)	Full video, screenshot, error with stack, URL and title, browser logs (console, JS errors, failed requests, HTTP ≥ 400), Node logs, Playwright trace and the page DOM at failure time (`reports/web/dom/`, used by the repairer)
API	Request and response (the `Authorization` header is hidden)	The same, plus the validation that failed
Mobile	Final screenshot (+ optional GIF)	Full video, screenshot, page source and device logs

Extra tools

command	what for
`npm run crawler [url]`	Generates a locators file by walking a page.
`npm run link-tester`	Looks for broken links and images in `BASEURL`.
`npm run explore`	Exploratory browser walk: links, images, errors, responsive layout and load times, with screenshots, videos and an HTML report (used by **web-explorer**).
`npm run accessibility`	WCAG 2.2 audit with axe-core, keyboard navigation and reflow at 320px, with screenshots, video and an HTML report (used by **accessibility-analyst**).
`npm run web-performance`	Lighthouse on mobile and desktop: Core Web Vitals, improvement opportunities and loading sequence, with an HTML report (used by **web-performance-auditor**).
`npm run pixel-perfect`	Visual regression against the reference: shifted, missing, new or restyled pixels and elements, with an HTML report (used by **pixel-perfect**).
`npm run cross-browser`	The web suite or the pages in Chromium, Firefox and WebKit, with a matrix and side-by-side comparison (used by **cross-browser**).
`npm run repair`	Diagnoses broken locators with the DOM saved at failure time, proposes, applies and verifies (used by **automation-repairer**).
`npm run emails`	Checks the emails in the Mailpit test mailbox against what's expected, with an HTML report (used by **email-verifier**).

`npm run translator`	Every page in every language: untranslated and cut-off text and formats, with an HTML report (used by **translation-analyst**).
`npm run ux`	Usability with Nielsen's 10 heuristics on desktop and mobile, with the 404 page, screenshots and an HTML report (used by **ux-evaluator**).
`npm run forms`	Validations of every field without submitting the form, with a field × test matrix, screenshots and an HTML report (used by **form-tester**).
`npm run permissions`	Role × route and role × endpoint matrix with screenshots and an HTML report (used by **permissions-tester**).
`npm run resilience`	Simulated API, network and resource failures from the browser, with a page × scenario matrix and an HTML report (used by **resilience-tester**).
`npm run monitor`	Read-only synthetic monitoring with history, a report and a Slack or Teams notification when something breaks or recovers.
`npm run functional-map`	Functional map of the site: pages, menus, forms with every field, buttons and messages, without submitting anything (used by **project-documenter**, in `input/documentation/`).
`npm run record`	Records a flow with Playwright codegen.
`npm run debug`	Runs the web suite in debug mode.

// guides

Performance testing

performance-mapper builds load, stress, soak and spike tests. It never uses default values: you define the load and the thresholds.

> ⚠ A load test generates real traffic. Run it only against test environments or with authorization from the system owners.

performance-mapper

> *Build a load test of the /pet endpoints for 50 users.*

1. **Guides you step by step**, one question at a time with a justified suggestion for each: goal and test type, tool, environment (and that it isn't production) and endpoints, concurrent users, ramp-up, duration and think time (it helps you calculate them from business volume), thresholds and profiles to run.
2. **Leaves the plan** in `output/performance/<US>/` and asks for confirmation.
3. **Recommends the tool** (the decision is yours): k6 for APIs (scales to thousands of users), Artillery + Playwright for browser flows (few users: each one is a Chromium) or JMeter if your team already uses it, you bring a `.jmx` or you need other protocols.
4. **Writes the script** once and validates it with a smoke run.
5. **Runs under load** only with your explicit confirmation for each run.
6. **Builds the performance report**: dashboard with verdict, observed capacity, metrics and charts per run, thresholds, p95 per endpoint, findings and recommendations.

Commands

```
npm run perf                                    # lists the scripts
npm run perf -- <script> smoke                  # validates the script with minimal load
npm run perf -- <script> load                   # target load (asks for confirmation)
npm run perf -- <script> stress --confirm       # no prompt: CI or with authorization
npm run perf -- <script> load --vus 20 --duration 2m   # k6 and JMeter: overrides the load
```

profile	what it does
`smoke`	Minimal load to validate that the script works.
`load`	Ramps up to the target load, holds it and ramps down.
`stress`	1x, 2x and 3x the target load: looks for the breaking point.
`soak`	Target load for a long time: memory leaks and degradation.
`spike`	Sudden jump to a peak and back: how it recovers.

Each run leaves in `reports/performance/<k6|artillery|jmeter>/` an HTML report with the verdict, requests, req/s, errors, p50/p95/p99 latencies, thresholds with the measured value, details per endpoint or step, response codes, checks and, in Artillery, the evolution over time. If a threshold isn't met, the command exits with an error.

// guides

Xray, QMetry, AIO Tests and Azure DevOps

test-manager and **results-publisher** work with Xray Cloud, Xray Server/Data Center, QMetry for Jira (QTM4J) and AIO Tests, with the same commands for all of them.

It's configured in the `.env`: `MANAGEMENT_TOOL` (`xray-cloud`, `xray-dc`, `qtm4j` or `aio`), `MANAGEMENT_PROJECT` (the Jira project key) and your tool's credentials, explained in `.env.example`.

test-manager

> *Upload the US-001 test cases to Xray in the 'US-001 Signup' folder, linked to PROJ-12, and create the Sprint 5 cycle.*

Creates the folder, uploads the test cases (manual ones from the `.xlsx` or BDD ones from the `.feature`), links them to the story, creates cycles and adds test cases to them. The first time it does a dry run (`--dry-run`). It saves traceability in `output/management/US-001-<tool>.json`: a test case is never uploaded twice.

Estimated time. Each test case is uploaded with its manual execution time: in the description and in the tool's time field — Jira's *Original Estimate* in Xray (if the project has time tracking), `estimatedTime` in QTM4J and `estimatedEffort` in AIO Tests. In Azure DevOps it travels in the JSON from `to_azure_devops.py` and as the tag `estimated-time-Xmin`. If the tool rejects the field, the test case is still created with the time in the description (`MANAGEMENT_NATIVE_TIME=0` turns it off). See the formula.

results-publisher

> *Upload the results of the last web run to cycle PROJ-60.*

Matches each scenario to its test case by the `@TC-XXX` tag and loads the status, a comment with the error and the evidence: always a screenshot, a GIF if one was generated and video only if it failed. It never assigns a result by approximation.

Equivalences

concept	xray	qtm4j	aio tests
Test case	Test issue (Manual or Cucumber)	Test case	Case (Classic or BDD)
Cycle	Test Execution	Test cycle	Test cycle
Case ↔ story	"Test" link (coverage)	Requirement link	jiraRequirementIDs
Cycle ↔ story	"Relates" link	Requirement link	jiraTaskIDs

Manual commands

```
python scripts/management/management.py --dry-run folder --path "US-001 Signup"
python scripts/management/management.py upload-cases --source <cases.xlsx> --folder "US-001 Signup"
--story PROJ-12 --trace US-001
python scripts/management/management.py create-cycle --name "Sprint 5" --story PROJ-12 --trace US-
001
python scripts/management/management.py extract-results --report <cucumber-report.json>
python scripts/management/management.py publish-results --cycle PROJ-60 --results <results.json> --
trace US-001
```

Folders go without a leading slash (in Git Bash, `/US-001` becomes a Windows path). Full reference in `scripts/management/README.md`.

Azure DevOps (Test Plans)

Azure DevOps doesn't go through `management.py`: it's integrated through its official MCP connector (`azure-devops`), with `ADO_ORGANIZATION`, `ADO_PAT` and `ADO_PROJECT` in the `.env`. The agents read the stories (with comments, tasks, parent and links) and the Wiki, and **test-manager** creates the *Test Cases* with their steps (built by `scripts/management/to_azure_devops.py`), links them to the story and adds them to a suite. Execution results aren't published to Azure DevOps yet.

> *Upload the test cases from cases-US-001.xlsx to Azure Test Plans, plan 'Sprint 5', suite 'US-001 Signup', linked to story 1234.*

// guides

Database checks

data-verifier checks what the tests don't see: what was saved, in what state and with what values. It supports **PostgreSQL, MySQL / MariaDB, SQL Server and MongoDB**, with several named connections.



On request

"Check whether user ana@test.com exists and what status it ended up in". It looks at the schema, builds the parameterized query and answers you.



Inside the tests, if you ask

Adds database steps to the web, API or mobile `.feature` files only when you ask; then they run with `npm test`, with no tokens.



Safe by default

Read-only, in transactions that are discarded; sensitive data masked. Writing must be enabled and each operation confirmed.

Configuration

```
DB_CONNECTIONS=main,payments              # the first one is the default
DB_MAIN_ENGINE=postgres               # postgres | mysql | mariadb | sqlserver | mongodb
DB_MAIN_HOST=localhost
DB_MAIN_PORT=5432
DB_MAIN_DATABASE=store
DB_MAIN_USER=qa_readonly
DB_MAIN_PASSWORD=...
DB_MAIN_WRITE=no                  # yes = allows writing with confirmation
DB_MAIN_PRODUCTION=no                 # yes = never writes
```

Steps for the tests

> **Only when you ask.** The mappers (web, API and mobile) don't add database checks on their own: if a test case mentions saved data, they leave it as a suggestion in the mapping report. data-verifier, for a one-off question, answers without touching the `.feature` files.

```
When I send a POST to "/users" with the body: ...
And I save the response field "id" as "userId"
Then the database has a record in "users" where "id" is "{userId}"
And the "users" record where "id" is "{userId}" has:
  | field      | value    |
  | status     | ACTIVE   |
  | deleted_at | (null)   |
And the database "payments" has 1 record in "transactions" where:
  | field   | value    |
  | user_id | {userId} |
```

There are also: `the database has no record in …`, `I query the database:` (SQL, or JSON in MongoDB), `the query returns N rows`, `the field "x" of the query is "y"` and `I save the field "x" of the query as "var"`. If the data is saved asynchronously, the steps retry up to `DB_WAIT_MS`. Each check leaves the query and the rows found in the report.

Without an agent

```
cd tools/k0lmena
npm run db -- connections                                  # lists and tests the connections
npm run db -- schema --table users                         # a table's columns
npm run db -- exists --table users --where email=ana@test.com
npm run db -- query --sql "SELECT id, status FROM users WHERE email = ?" --param ana@test.com
```

> ⚠ To prepare or clean up data, the connection needs `DB_<NAME>_WRITE=yes` and the agent shows the statement and asks for confirmation before each operation. With `DB_<NAME>_PRODUCTION=yes` it never writes. Recommended: a database user with read-only permissions.

// guides

Exploratory testing

web-explorer walks your site (the `APP_URL` in the `.env` or the URL you give it) and looks for what breaks without anyone noticing. First an **automatic walk** with a script, without spending tokens per page; then it explores by hand with Playwright MCP whatever needs judgment.

what it checks	how
Links and images	Broken internal and external links (separately, the ones that can't be verified because the site blocks robots), images that don't load and images without `alt`.
Errors	JavaScript errors, console errors and failing resources (scripts, CSS, fetch with 4xx/5xx or no response).
Responsive	At 360×740, 390×844, 768×1024, 1366×768 and 1920×1080: horizontal scroll and which element overflows, text under 12px, small and crowded touch targets (WCAG 2.5.8) and missing `meta viewport`.
Performance	Load time, time to first byte, DOM ready, requests and weight of each page; slow ones are findings.
Manual exploration	Menus (the mobile one too), forms without submitting, searches, the 404 page, reloading mid-flow. With a mission and a time box.

web-explorer `uses tokens`

> *Run exploratory tests on the site in the .env.*

1. **Walks** the site with `npm run explore`: screenshots of every page and every resolution with the problems **marked in red**, and a video of the walk and of each resolution.
2. **Reviews** the findings: retries the doubtful ones, looks at the screenshots and adjusts the severity with its reason.
3. **Explores by hand** with Playwright MCP what an automatic walk doesn't see.
4. **Builds the HTML report**: indicators, charts, every finding with its screenshot (click to enlarge), load times, responsive matrix, videos, broken links, details per page and recommendations.

```
npm run explore -- --url https://staging.my-app.com --max-pages 50 # no agents, no tokens
npm run explore -- --resolutions 360x740,768x1024,1366x768 --video no
```

It only navigates with GET: it doesn't submit forms and doesn't open or check logout or delete links. Everything stays in `output/exploratory/<US or site>/` ; the previous run moves to `history/` (the last 5 are kept) and the report compares against it.

// guides

Accessibility

accessibility-analyst audits your site against **WCAG 2.2** (level AA by default): first an automatic audit, without spending tokens per page, and then a manual review of what a tool can't judge.

what it checks	how
WCAG rules	**axe-core** on every page, with the texts in English: contrast, alternative texts, form labels, button and link names, language, structure, ARIA. Best practices too.
Keyboard	Walks the page with Tab: elements that receive focus without a visible indicator (2.4.7), focus traps (2.1.2) and a skip-to-content link (2.4.1). Shows the focus order.
Reflow	At 320px wide there must be no horizontal scroll (1.4.10, equivalent to a 400% zoom).
Manual review	With Playwright's accessibility snapshot (what a screen reader "hears"): whether alternative texts describe, headings and regions, forms and error messages, menus and modals with the keyboard, zoom, color and motion.

accessibility-analyst `uses tokens`

> *Review the accessibility of the site in the .env.*

1. **Audits** with `npm run accessibility` : screenshots of every page with the failing elements **marked in red**, each element separately and a video of the walk.
2. **Reviews** the findings: confirms with the screenshots, adjusts the severity and decides what axe couldn't.
3. **Reviews by hand** the most important flows with Playwright MCP.
4. **Builds the HTML report**: compliance, failed WCAG criteria with their name and level, every problem with its elements, screenshots and how to fix it, keyboard and focus order, and recommendations.

```
npm run accessibility -- --url https://staging.my-app.com --level AA # no agents, no tokens
npm run accessibility -- --pages https://my-app.com/signup,https://my-app.com/login --story US-001
```

An automatic audit detects between 30 and 50% of accessibility problems: the agent never says the site "complies", it says what doesn't comply and what was left unreviewed. Everything stays in `output/accessibility/<US or site>/` ; the previous run moves to `history/` (the last 5 are kept) and the report compares against it.

// guides

Web performance

web-performance-auditor measures how fast each page loads for one person and why, with **Lighthouse** on mobile and desktop. It isn't a load test with many users: that's **performance-mapper**.

web-performance-auditor `uses tokens`

> *Check the speed of the site in the .env.*

1. **Measures** each page with `npm run web-performance` : scores, Core Web Vitals (LCP, CLS and TBT) with Google's thresholds, FCP and Speed Index.
2. **Finds** the improvement opportunities with the estimated savings and the resources causing them, the LCP element, weight by resource type and third-party code.
3. **Builds the HTML report** with the loading sequence in screenshots, Lighthouse's native report for each page and prioritized recommendations.

```
npm run web-performance -- --url https://staging.my-app.com --max-pages 5 # no agents, no tokens
npm run web-performance -- --pages https://my-app.com/,https://my-app.com/login --repetitions 3
```

This is lab data (a simulated load under fixed conditions): it's for finding and comparing problems, not for claiming how each person experiences it. To compare before and after, use `--repetitions 3` . Requires Node 22.19 or later.

// guides

Failure analysis

When `npm test` fails, **failure-analyst** separates real bugs from broken, flaky, data or environment

tests, so you don't review failure by failure.

failure-analyst `uses tokens`

> *Analyze why npm test failed.*

1. **Classifies** each failure with `scripts/analyze_failures.py` : possible bug, locator or wait, environment, test data, automation error or undefined step, and recovers the screenshot, video and trace.
2. **Retries** the failed ones (with your ok) and adds the retry: what failed and then passed is marked as flaky.
3. **Confirms** the cause with the evidence, writes the bug reports for the real ones and proposes how to fix the rest.

```
python scripts/analyze_failures.py                      # the last npm test run
python scripts/analyze_failures.py --add-retry   # after re-running the failed ones
```

Re-running the failed tests runs the suite against the environment: it's only done with your confirmation. Changes to the automation are made by the mappers.

// guides

Server logs

log-analyst reads the application logs and crosses them with the `npm test` run: which errors happened, when, and which server error was behind each failed scenario.

log-analyst `uses tokens`

> *Why did TC-004 fail on the server side? The logs are in input/logs/.*

It understands app logs (ISO, `YYYY/MM/DD`), nginx and Apache access logs, JSON lines, syslog and `.gz` . It groups errors by signature (ignoring numbers and ids), attaches their stack trace and places them in time. The k0lmena steps record the start and end of each scenario, so every failure gets the server errors that happened while it ran. Delivers `logs-report.html` with errors and warnings per minute, the signatures and each failed scenario with its context.

```
python scripts/analyze_logs.py --logs "input/logs/*.log" --run web
python scripts/analyze_logs.py --logs input/logs/app.log --from 2026-10-11T10:00 --until 2026-10-11T11:00
```

Without `--from` or `--until` it analyzes the run's window plus 5 minutes before and after. `input/logs/` is not versioned. Downloading logs from a shared server is confirmed first. Output in `output/logs/<name>/`.

// guides

Coverage and traceability

coverage-analyst answers “what's left to test?”: it crosses criteria, test cases, automation, management, results and bugs for each story and marks where the chain breaks.

coverage-analyst `uses tokens`

> *What's left to test in US-001?*

1. **Crosses**, with `scripts/analyze_coverage.py`, the criteria (story, context sheet, coverage tables or Jira), the manual and BDD test cases, the k0lmena automation, the traceability with Xray/QMetry/AIO, the latest results and the bugs.
2. **Marks the gaps**: criteria without test cases, test cases without a criterion, not automated, blocked or not run, failed, not uploaded to the test management tool and open missing information.
3. **Builds the HTML report** with the funnel from story to result, coverage per criterion, the case-by-case matrix and what to do about each gap.

```
python scripts/analyze_coverage.py --story US-001
python scripts/analyze_coverage.py                    # every story in the repo
```

Read-only: it doesn't create test cases or automate. The `@US` + `@TC` pair is what traces each test case; a scenario without `@TC` shows up as a gap.

// guides

Visual regression (pixel perfect)

pixel-perfect detects **what changed visually** between the accepted version of your site (the reference) and the current one: what shifted and by how many pixels, what changed size, text or color, what's missing and what appeared.

what it compares	how

Pixels	Percentage of the screen that changed and the regions with changes, with the difference map in red.
Elements	Position and size of every heading, link, button, field and image: shifted (with the px), missing, new and with different text.
Shifted blocks	If many elements moved the same way, it's a single cause (a new banner pushed everything down): it reports the block and the probable cause.
Styles	Background color, text color, font size and weight.

pixel-perfect `uses tokens`

> *Run a visual regression of https://staging.your-app.com and tell me if anything shifted.*

1. **The first run creates the reference**: a stable screenshot of every page at every resolution (no animations, fonts and images loaded and dynamic areas hidden).
2. **The following runs compare** against it, pixel by pixel and element by element.
3. **Separates the intentional** (what the story asked for) from regressions.
4. **Builds the report**: reference/current slider comparator, difference map, close-ups of the regions and the before and after of every finding.

```
npm run pixel-perfect -- --url https://staging.your-app.com --resolutions 390x844,1366x768 # no agents, no tokens
npm run pixel-perfect -- --hide ".banner,.date"      # areas that change on their own
npm run pixel-perfect -- --accept all                  # current becomes the reference (with your ok)
```

The reference can also be a Figma design exported as PNG. Everything stays in `output/pixel-perfect/<US or site>/` : `reference/` is kept and, on every comparison, the previous run moves to `history/` (the last 5 are kept) and the report compares against it.

// guides

Cross-browser

cross-browser finds what works or looks different depending on the browser: **Chromium** (Chrome

and Edge), **Firefox** and **WebKit** (Safari's engine), on desktop and mobile.

Suite mode

Runs the k0lmena web suite once per browser (each in its own folder) and builds the scenario × browser matrix: passes in all, fails only in some or fails in all.

Pages mode

No automation needed: walks the pages in each browser and compares them against the base: JavaScript and console errors in only one, overflow, missing or shifted elements and different styles.

cross-browser `uses tokens`

> *Test US-003 in Chrome, Firefox and Safari.*

The report brings the matrix with each browser's screenshot, the pages side by side with the slider comparator, and every difference with its error and probable cause. What fails in all of them isn't a browser issue: it hands it to **failure-analyst**.

```
npm run cross-browser -- --story US-003                                  # suite mode (runs against the environment: with your ok)
npm run cross-browser -- --mode pages --url https://staging.your-app.com # GET only
cd tools/k0lmena && npx playwright install firefox webkit        # if the browsers are missing
```

WebKit is Safari's engine and finds almost everything, but it doesn't replace a real Safari on a Mac or iPhone. Output in `output/cross-browser/<US, site or suite>/`; the previous run moves to `history/` (the last 5 are kept) and the report compares against it.

// guides

Automation repair

When a test fails because **the screen changed** (a button with another name, a selector that now finds several elements), **automation-repairer** proposes the new locator, applies it and verifies that the test passes again. If it fails because the application has a bug, it doesn't touch the test.

diagnosis	what it does

No longer exists	Looks for the most similar element (role, name, text, label, placeholder, data-testid) and proposes the most robust locator that finds **a single** element. In a verification step (Then) it flags it as a possible bug.
Matches several	Tries `exact: true` or scopes it to each element's container (the product card) and lets you choose.
Exists but isn't visible	A previous step is missing (opening a menu) or the app doesn't show it.
Exists and is visible	The locator is fine: it proposes the right wait.

automation-repairer `uses tokens`

> *Repair the US-001 tests that failed because of locators.*

1. **Analyzes** the failures of the last `npm test` (or from failure-analyst) against the DOM that k0lmena saved at failure time, without touching files.
2. **Applies** the safe repairs (or the ones you confirm), with a backup copy and `changes.diff`.
3. **Verifies** by running only the repaired scenarios (with your ok) and builds the report with the diagnosis, the change and the screenshots.

```
npm run repair -- --story US-001                  # diagnosis and proposals
npm run repair -- --apply safe --story US-001     # or --apply R-01,R-03
npm run repair -- --verify --story US-001         # re-runs the repaired ones
```

Since this version, k0lmena saves the page DOM at failure time (`reports/web/dom/`) and the viewport was fixed: if `VIEWPORT_WIDTH` and `VIEWPORT_HEIGHT` are empty, the window uses 1366×768 (before it stayed at 0 and clicks failed). Output in `output/repairs/<US or suite>/`.

// guides

Test case selection for a change

test-case-selector answers “what should I test with this change?” without running the whole regression: it crosses what changed with all the test cases and delivers a prioritized, explained list ready to run.

input	what it uses
The change	A git range (files, commits with their Jira keys and touched functions), a PR's files, Jira and Azure DevOps stories or issues, or the release notes.
The inventory	Manual (Excel), BDD and k0lmena automated test cases, with their last result, the flaky ones and open bugs.

The result	Must-run, recommended and minimal regression (smoke and Critical priority), with the reason, estimated time, savings and the changes no test case covers.

test-case-selector `uses tokens`

> *Which test cases should I run for PR #42? It touches card payment (PROJ-55).*

Delivers `selection-report.html` with ready-to-copy commands (bash and PowerShell) for what's automated and `selection.csv` with the manual ones. It doesn't run the tests: the automated ones run with `npm test`, with no tokens.

```
python scripts/select_cases.py --name release-2.4 --git main..feature/payments --repo ../my-app
python scripts/select_cases.py --name pr-42 --stories US-003 --budget-min 60
TAGS="(@US-003 and (@TC-001 or @TC-003))" npm test -- web   # the command the report builds
```

Optional: `input/component-map.json` maps code folders to stories to refine the selection. Output in `output/case-selection/<name>/`.

// guides

Test case review and estimated time

case-reviewer reviews existing test cases (manual in Excel or BDD scenarios) before running, uploading or automating them, and calculates how long they take to run by hand.

case-reviewer `uses tokens`

> *Review the US-001 test cases and tell me how long they take to run.*

A script reviews each case (title, preconditions, actionable steps, verifiable expected result, data, priority, traceability) and scores it from 0 to 100; it detects duplicates and near-duplicates and stories without negative or edge cases. The agent adds what needs judgment and proposes an improved version of the worst ones. Delivers `review-report.html`; if you ask, the corrected spreadsheet goes into a new file and never overwrites the original.

```
python scripts/review_cases.py --story US-001
python scripts/review_cases.py --source output/test-cases/manual/cases-US-001.xlsx,output/test-cases/bdd/US-001-signup.feature
python scripts/estimation.py output/test-cases/manual/cases-US-001.xlsx   # only the time
```

How the estimated time is calculated

Each test case (manual or BDD) carries the time a person who knows the application takes to run it end to end. It's calculated by `scripts/estimation.py` and appears in the spreadsheet (the **Estimated Time (min)** column with the total), in the `.md`, in the test management tool, in the test case selection and in this review.

```
case time = (setup + Σ steps + logging) × factor   # rounded up to 0.5 min, minimum 1
step time = action + data + verification
```

part	how it's calculated
Action	By verb: navigate 0.5 · click 0.25 · fill in 0.3 per field · log in 1 · upload or download a file 1 · wait for an email or a process 2 · query a database, an API or logs 2 · read 0.5 · other 0.5 min.
Data	0.15 min for each extra piece of data in the step.
Verification	0.5 min if there's an expected result (or it's a `Then`) + 0.25 for each extra check (up to 4).
Setup	1 min + 0.5 per precondition (up to 3) + 2 if data or users have to be created.
Logging	0.5 min + 0.1 per step (marking the result and saving the evidence).
Factor	1.1 for negative or edge cases; `ESTIMATION_FACTOR` in the `.env` multiplies everything (for example 1.3 for someone new to the application).

If the spreadsheet already has a value in **Estimated Time (min)**, that one wins: what the person entered is respected. Full formula, with examples, in `templates/time-estimation.md`. Review output in `output/case-review/<US or name>/`.

// guides

Charters and exploratory sessions

charter-generator plans session-based exploratory testing: each charter says what to explore, with which focus and heuristics, with which data and in how much time. Afterwards it builds the report with what each session found.

charter-generator `uses tokens`

> *Build charters to explore the checkout in release 2.4.*

Delivers the session sheets (`charters.html` and `.md`) and a notes template per session in `sessions/CH-XX.md`. Whoever explores takes notes with prefixes (`BUG:` `QUESTION:` `IDEA:` `RISK:` `COVERED:` `NOT COVERED:` `NOTE:` and `Time:`) and the report comes out of those notes: bugs, questions, risks, time split and covered areas.

```
python scripts/generate_charters.py --name US-003            # session sheets
python scripts/generate_charters.py --name US-003 --report   # sessions-report.html from sessions/
```

If you ask it to explore itself, it uses Playwright MCP and fills in the session notes. Output in `output/charters/<US or name>/`.

// guides

Email

email-verifier checks that the emails your application sends (sign-up, confirmation, password recovery, orders, notifications) **arrive correctly**: to the right person, with the right content and looking good in email clients. It uses **Mailpit**, a test mailbox that catches the test environment's emails without delivering them to anyone.

what it checks	how
What's expected	For each case: recipient, sender, subject, texts it contains (and doesn't), links and attachments. Flags the ones that didn't arrive.
Content	Unreplaced variables (`{{name}}` , `undefined` , `null`), subject, plain-text version and size (Gmail clips from 102 KB).
Links and images	Broken links, links to `localhost` or without https; images without alternative text or pointing to local paths.
How it looks	HTML compatibility with Outlook, Gmail, Apple Mail and others (Mailpit's check) and screenshots on desktop (700 px) and mobile (375 px).

email-verifier `uses tokens`

> *Check that the welcome email reaches ana@test.com.*

1. **Start Mailpit** and point the test app's SMTP to `localhost:1025`.
2. **Trigger the email** (yourself, `npm test` or the agent with your ok: it creates data).
3. **It checks the mailbox** against what's expected for each case and builds the report with a view of each email.
4. **If you ask**, it adds email steps to the k0lmena `.feature` files, which run without tokens.

```
docker compose -f tools/mailpit/docker-compose.yml up -d      # web and API on :8025, SMTP on :1025
npm run emails -- --to ana@test.com --expect input/emails/expect-US-001.json
npm run emails -- --to ana@test.com --wait 1 --timeout-s 90   # waits for it to arrive
```

```
Then an email arrives at "ana@test.com" with the subject "Confirm your account"
And the email contains "Hello Ana"
And the email does not contain "undefined"
When I save the link from the email that contains "/confirm" as "confirmationLink"
```

> ⚠ Test mailboxes only: never real inboxes or the production mail server. Emptying the mailbox (`--clean-mailbox`) is confirmed first. Output in `output/emails/<US or mailbox>/` ; the previous run moves to `history/` (the last 5 are kept) and the report compares against it.

// guides

Translations and languages

translation-analyst walks every page in every language (the first one is the reference) and finds what's left to translate or looks wrong in another language. It proposes the translation for what's missing.

what it looks for	example
Untranslated text	A paragraph in Spanish in the English version, or identical to the reference; also in placeholders, alternative texts and `aria-label` .
Visible keys	`checkout.add_to_cart` , `MISSING_TRANSLATION` .
Broken characters	`CafÃ©` instead of `Café` .
Cut-off text	A button that leaves no room for a longer translation.
Formats	`1.234,56` in the English version; “octubre” in an English date; the wrong `lang` attribute.

translation-analyst `uses tokens`

> *Review the site's Spanish and Portuguese translations.*

It switches language with the site's `hreflang` , with `--language-url` (for example `https://your-app.com/{short_language}/`) or by asking the browser for the language. The report brings the percentage translated per language, every page in all languages side by side and every problem with its crop.

```
npm run translator -- --url https://staging.your-app.com --languages en-US,es-AR,pt-BR # no agents, no tokens
npm run translator -- --url https://your-app.com --language-url "https://your-app.com/{short_language}/"
```

Language detection is a heuristic: the agent confirms each case and discards proper names and brands. Output in `output/translator/<US or site>/`; the previous run moves to `history/` (the last 5 are kept) and the report compares against it.

// guides

Usability

ux-evaluator reviews what confuses, slows down or misleads the person, with **Nielsen's 10 heuristics**: an automatic walk through the pages and a manual one through the important flows.

ux-evaluator `uses tokens`

> *Evaluate the checkout's usability.*

The automatic walk checks, on desktop and mobile, what can be measured: the 404 page (whether it helps you get back), very long menus or menus without the current section marked, different names for the same action, links that open another tab without warning, icon-only buttons, fields with the placeholder as their only label, technical messages in view, the main action below the fold on mobile and more. The agent walks the flows with Playwright MCP and adds what only shows up when using them. Delivers `ux-report.html` with a score per heuristic, every finding with its screenshot and recommendations.

```
npm run ux -- --url https://staging.your-app.com   # no agents, no tokens
npm run ux -- --url https://your-app.com/checkout --max-pages 4 --resolutions 1440x900,390x844
```

It doesn't advance in a flow that creates data (an order, an account) without confirmation. Output in `output/ux/<US or site>/`; the previous run moves to `history/` and the report compares against it.

// guides

Forms

form-tester tests the validations of every field in a form **without submitting it**: required fields, lengths, formats, limits and error messages.

form-tester `uses tokens`

> *Test the sign-up form's validations.*

It finds the forms, works out what each field is (type, label, `required`, `maxlength`, `min` / `max`, `pattern`) and tries what applies: empty, only spaces, longer than the maximum, invalid emails and phone numbers, out-of-range numbers, names with accents and ñ. It flags **accepted invalid values** and **rejected valid ones**, fields without a label and messages that don't say what to fix. If the story defines rules, the agent compares against them. Delivers `forms-report.html` with the field × test matrix and screenshots with the errors in view.

```
npm run forms -- --url https://staging.your-app.com/signup   # no agents, no tokens
npm run forms -- --url https://staging.your-app.com/signup --submit-empty   # only with confirmation
```

⚠ It never submits a form with data: it blocks every submission that isn't GET. The only thing it can submit, with confirmation, is the empty form (`--submit-empty`). Output in `output/forms/<US or site>/`.

// guides

Role permissions

permissions-tester checks that each role sees and can do only what it's supposed to.

permissions-tester `uses tokens`

> *Test the admin, operator and guest permissions on https://staging.your-app.com.*

It logs in with each role in the `.env` (`PERMISSIONS_ROLES`, `PERMISSIONS_<ROLE>_USER` / `_PASSWORD` / `_TOKEN`) plus anonymous, walks the site, opens each route with each role **by direct URL** and requests the endpoints with each one's session and token. It builds the role × route and role × endpoint matrix (opens, denies, asks for login), which menus and buttons each role sees and the differences from the expected rule (project-documenter's `statuses-and-permissions.md` or the story). Typical findings: protected routes open without login, another role's URLs, visible buttons that then deny access and APIs without authentication.

```
npm run permissions -- --url https://staging.your-app.com --endpoints "GET /api/users"
```

GET only: it doesn't make clicks with side effects or submit forms without confirmation. It doesn't store credentials. Output in `output/permissions/<US or site>/`.

// guides

Resilience

resilience-tester answers what the person sees when something fails underneath.

resilience-tester `uses tokens`

> *Test what happens on the product list if the API goes down or the network is slow.*

From the browser it intercepts the network and simulates, page by page: an API returning 500, 503 or 404, timeout, slow network, offline, empty responses, broken JSON or null fields, and static files (CSS, JS, images, fonts) that don't load. It detects blank screens, silent failures, endless spinners, technical messages, JavaScript errors, retries and whether the page recovers when the network comes back. Delivers the page × scenario matrix with a screenshot of each.

```
npm run resilience -- --url https://staging.your-app.com/products --scenarios api-500,timeout,offline
```

It doesn't generate load or attack the server: it only intercepts what the browser requests. Output in `output/resilience/<US or site>/`.

// guides

Bug triage and questions

bug-triager `uses tokens`

> *Is BUG-008 already filed in Jira? · Triage the bugs in output/bug-reports.*

Before a bug is filed it looks for candidates in Jira (JQL), Azure DevOps (WIQL), the local bugs and the repository issues, closed ones included, and with `scripts/find_duplicates.py` classifies them as duplicate, **possible regression**, related or different. It suggests severity and priority with its reasoning, component, owner (from `CODEOWNERS` and commits) and the story to link. Creating, linking or commenting in the tool happens only with your confirmation. Output in `output/triage/`.

project-consultant `uses tokens`

> *What happens if a blocked user tries to pay?*

It answers product questions using only the project documentation (`input/documentation/`, context sheets and artifacts), citing the original source of every fact and separating what the story says from what the code does today. If it isn't documented, it says so and proposes the `MI-XX`.

// guides

Run history

Every tool keeps its **last 5 runs** with their evidence, so you can compare before and after a fix without losing anything.

where	what it keeps
`output/<tool>/<US or site>/history/<date-time>/`	The JSON, the analysis, the report and the evidence (screenshots, videos, crops) of each previous run: its report still opens the same way.
`tools/k0lmena/reports/<suite>/history/`	The Cucumber report and the evidence of each previous `npm test` (screenshots, videos, traces and the DOM at failure time).

New

What shows up in this run and wasn't in the previous one.

Resolved

What was there and no longer shows up.

Evolution

The main indicators of each run over time, with the link to each one's report.

Every report (exploratory, accessibility, web performance, visual regression, cross-browser, failures, coverage, repairs, selection, email, translations, case review, charters, usability, forms and logs) includes the **Run history** section. To keep more runs: `RUN_HISTORY=10` in the k0lmena tools' environment.

// guides

Quality dashboard and report portal

quality-dashboard `uses tokens`

> *Build the quality dashboard for sprint 12 (October 1 to 14).*

It gathers what the tools already left (executions, `npm test` and its history, test cases, coverage, bugs, flaky tests, accessibility, web performance, load performance and security) and builds a dashboard with pass rate and trend, % automated, coverage, bugs by severity and story, manual execution minutes, alerts and a **management summary**. Whatever has no data says "No data".

```
npm run dashboard -- --name sprint-12 --from 2026-10-01 --until 2026-10-14
npm run portal -- --open   # output/index.html
```

The **report portal** (`output/index.html`) shows the latest report from each tool with its date, verdict and the previous run. It has a search box, filters by tool, story, status and date (remembered between visits) and a **by story** view, with all the reports for a story together, from any tool. It regenerates itself every time a report is built and opens with a double click.

// guides

Notifications and synthetic monitoring

Notifications to Slack and Teams

`scripts/notify.py` posts through a webhook (`SLACK_WEBHOOK_URL` with Block Kit and `TEAMS_WEBHOOK_URL` from Teams Workflows with an Adaptive Card). The triager (critical bug), the dashboard and the monitoring use it, and it can be called after `npm test`. With no webhook configured it doesn't send anything.

```
npm run notify -- --title "Sprint 12 regression" --text "3 failed" --level error --link https://…
npm run notify -- --from-cucumber tools/k0lmena/reports/web/cucumber-report.json
```

Synthetic monitoring

`npm run monitor` runs a **read-only** smoke check against any environment: status and time of each URL, expected text, TLS certificate about to expire and, if you want, k0lmena scenarios tagged `@synthetic`. It compares with the previous run and **notifies only when something breaks or recovers**. It's scheduled with `.github/workflows/monitoring.yml`, cron or the Windows Task Scheduler.

```
npm run monitor -- --urls https://your-app.com,https://your-app.com/login --name production
```

// guides

Agent evals

The tests cover the scripts; the **evals** measure the quality of what the agents generate.

Each case in `evals/cases/` gives an agent a story, a contract, an observation or a repository with **deliberate traps** (ambiguities, a contradiction between description and comment, undefined messages, limits, a fake secret) and verifies with deterministic checks that it marks the Missing information, applies boundary values, respects the templates and doesn't make things up. It runs in an isolated workspace with `claude -p` and leaves a report with the score per agent and the **regressions** against the previous run.

```
npm run evals -- --list
npm run evals -- --run --agent manual-case-generator
npm run evals -- --verify-only <folder> --case manual-cases-signup-story   # without running the agent again
```

> ⚠ Running the evals spends tokens (each case is a full conversation, with a cap per case). Use them before and after changing an agent or a skill.

// guides

Token usage

How much each agent spends and where it pays to automate.

`scripts/token_metrics.py` reads this project's Claude Code transcripts (`~/.claude/projects/`): from each model response it takes the input, output and cache tokens and the model, **never the text**. It attributes usage to each agent (subagents by their type; the rest is the main conversation) and builds a report with the estimated cost per agent, day and model, the percentage read from cache and **opportunities**: the agents that spend the most, the ones used many times for the same thing (candidates for automation in k0lmena, which then runs without tokens) and low caching. It also adds the cost of the evals. The quality dashboard shows the summary.

```
npm run metrics                                          # the whole history
npm run metrics -- --name october --from 2026-10-01   # one period
```

The cost is an estimate based on the script's per-million-token price table (`PRICES`, with its date and source); it doesn't replace actual billing. Models not in the table are reported as "no price".

// guides

Onboarding

So that someone who just joined becomes productive in this project quickly.

onboarding-guide `uses tokens`

> *Build the onboarding for Ana, who's joining as a manual QA.*

It reads the project documentation (starting with the index built by project-documenter), runs `npm run doctor`, looks at the active connectors and what already exists (stories, test cases, automation, reports) and builds a plan: the product with its modules, flows, roles and glossary (with the source of every fact), the state of their environment, the agents that suit their role with **example requests from this project**, the tasks for the first weeks with their definition of done, the questions for the team about what isn't documented and a **checklist** that gets ticked in the browser. It comes out in `output/onboarding/<person>/` as HTML and Markdown.

// guides

Web security

security-analyst reviews your application's security with **OWASP ZAP** (free and open source) in **passive** mode: it walks the site and analyzes the responses without sending attacks or modifying data.

Only your own sites or sites with written authorization are scanned, declared in `SECURITY_AUTHORIZED_URLS` in the `.env`. A passive scan doesn't replace a penetration test.

security-analyst

> *Review the security of https://staging.my-app.com for US-001.*

1. **Confirms** the URL (it has to be authorized) and how long to crawl.
2. **Scans** with the ZAP baseline in Docker (open Docker Desktop first).
3. **Analyzes** the findings: prioritizes them, discards false positives with their reason and explains them in English.
4. **Builds the HTML report**: findings by risk, where each one appears, evidence, how to fix it and recommendations.

```
python tools/zap/scan.py --url https://staging.my-app.com --story US-001
```

Output in `output/security/<US or host>/<date>/`. Details in `tools/zap/README.md`.

web-pentester

> *Try to hack https://staging.my-app.com and report what you find.*

1. **Confirms** the URL (authorized), the active-pentest authorization (`PENTEST_AUTHORIZED=yes`), the environment (no production without permission) and the scope.
2. **Active scan** with the ZAP full scan (sends attacks) plus **manual OWASP Top 10 verification** with Playwright: broken access control and IDOR, authentication and session, injection, XSS, misconfiguration, CSRF/SSRF.
3. **Confirms** each finding (discards false positives) and measures the real impact on your site.
4. **Builds the HTML report** (active mode) and a `pentest-report.md` with steps to reproduce and how to fix.

```
python tools/zap/scan.py --url https://staging.my-app.com --story US-001 --active --confirm
```

Authorized ethical hacking: only URLs in `SECURITY_AUTHORIZED_URLS`, with `PENTEST_AUTHORIZED=yes`, on a site you own or have written permission for, and never against production without authorization. Not destructive.

// guides

Streaming and cameras

Five agents for video and security-camera QA: verify the stream plays well, control the camera over **ONVIF**, check image quality, review recordings and analyze device security. They cover IP cameras (RTSP/ONVIF), live streams (RTMP/HLS/DASH), WebRTC served over HTTP and VOD

playback.

> ⚠ Only your own devices or streams, or ones you're authorized in writing to test, declared in `STREAMING_AUTHORIZED_SOURCES`, `ONVIF_AUTHORIZED_HOSTS` or `AUTHORIZED_CAMERAS` in the `.env`. Everything is read-only (no recording or re-streaming), except PTZ, which moves the camera and is confirmed.

streaming-tester

> *Test the camera rtsp://cam.local/stream and check stability.*

1. **ffprobe** reads the properties: codec, resolution, FPS, bitrate, audio and keyframes (GOP).
2. **ffmpeg** measures startup latency, continuity (dropped frames, errors) and, with `--stability`, a soak with reconnection; with `--connections`, a bounded load test.
3. **Builds the HTML report** with properties, continuity and findings by severity.

```
npm run streaming -- --source rtsp://cam.local/stream --stability 60 --connections 10 --confirm
```

onvif-tester

> *Test the camera 192.168.1.50 over ONVIF.*

1. **Reads** device info, capabilities, profiles and stream URIs, snapshot and events.
2. **Checks** the clock (drift vs. real time), timezone, NTP and firmware against the site standard.
3. **PTZ** optional (`--ptz --confirm`): moves the camera minimally and stops it. Returns the RTSP URI for streaming-tester.
4. **Live events** (`--events N`): subscribes via pull-point and waits N seconds for a real motion event (move in front of the camera).

```
npm run onvif -- --host 192.168.1.50 --confirm
```

image-quality-analyst

> *Does the camera look OK? Check if it's blurry or moved.*

1. **OpenCV** detects black frames, blur and freeze (and, with a reference, whether the camera moved).
2. **With vision**, the agent looks at the capture and judges a dirty or blocked lens, backlight and IR that doesn't switch at night.
3. **Builds the report** with the capture, metrics and findings per camera.

```
npm run image-quality -- --source rtsp://cam.local/stream --reference ref.jpg --confirm
```

recording-verifier

> *Is the NVR 192.168.1.60 recording and meeting 30 days of retention?*

1. **Queries** over ONVIF (Recording/Search) the recorded range, the count and the tracks.
2. **Detects** gaps and compares the covered days with the expected retention; warns if the last recording is old.
3. If the device doesn't support it over ONVIF, it reports so (it may need the NVR API).

```
npm run recordings -- --host 192.168.1.60 --retention 30 --confirm
```

camera-security-analyst

> *Review the security of cameras 192.168.1.50 and .51.*

1. **Passive scan** of ports and insecure services: Telnet, FTP, HTTP without TLS, RTSP without authentication.
2. **Reads** the firmware (if ONVIF credentials are given) and reminds you to review its CVEs. It doesn't test credentials or attack.
3. **Builds the report** with ports, insecure services and findings per camera.

```
npm run camera-security -- --host 192.168.1.50,192.168.1.51 --confirm
```

Requirements: **ffmpeg** (streaming and image quality) and, for ONVIF and quality, `onvif-zeep` and `opencv-python-headless` (installed automatically). Output in `output/streaming/`.

// guides

MCP connectors

They connect Claude Code to external systems. Claude Code starts right away: Playwright and Atlassian (Jira and Confluence) come active, and each QA enables the rest with `npm run connector`. No secrets: credentials are read from the `.env`, and `.env.example` has a commented block per tool so you fill in only your team's.

connector	what for	authentication
`playwright`	Browser for the E2E runner and the web-mapper (headed and headless).	Active
`atlassian`	Jira and Confluence: stories, criteria and functional documentation directly. With a token, Bitbucket Cloud too.	Active · OAuth or API token
`figma`	Designs from a link to a frame or file: texts, states and components. Requires a Full or Dev seat.	Account (OAuth)
`appium-mcp`	Used by mobile-mapper to walk the app.	`ANDROID_HOME`
`qmetry` · `qtm4j`	Query QMetry from the chat.	API key
`aio-tests`	Query AIO Tests from the chat.	Token
`azure-devops`	Azure DevOps: stories, Wiki, Azure Repos and Test Plans test cases.	PAT
`linear` · `clickup` · `monday` · `asana` · `trello` · `notion` · `youtrack` · `shortcut`	Stories and documentation if your team uses neither Jira nor Azure DevOps (official MCP servers).	Account (OAuth); YouTrack with a token; Asana with your own app
`github` · `gitlab`	The app repository for project-documenter (code, PRs and commits, read-only).	Token (GitLab: OAuth too)
`k0lmena-tmt`	Upload and query test cases in k0lmenaTMT.	Personal token

01

Fill in the .env

If it uses a token, put it in the `.env` at the root. Atlassian and Figma are authorized with your account from `/mcp`.

02

Enable it

`npm run connector -- activate azure-devops` (only for you; `npm run connector` lists the available ones).

03

Verify

Restart Claude Code and check the connection with `/mcp`.

> *Generate the PROJ-12 test cases using the Confluence page 'Billing rules' and this Figma design.*

Xray doesn't need MCP: it uses `scripts/management/`. Full guide in `CONNECTORS.md`.

// reference

Conventions

element	format
Stories	`US-001`
Acceptance criteria	`AC1` · `AC2`
Manual test cases	`TC-001`
API test cases	`TC-API-001`
Bugs	`BUG-001`
Missing information	`MI-01` · `@missing-info @MI-01`

scale	values
Severity and priority	Critical · High · Medium · Low
Test case status	N/A · Pending · In progress · Passed · Failed · Blocked

Tags in kOlmena

where	tags
Feature	The story and the type: `@US-001 @web` · `@api` · `@mobile`
Scenario	The test case ID (`@TC-001` , `@TC-API-001`) and `@smoke` if it's critical
Exclude	`@blocked` : a step couldn't be verified and the scenario doesn't run

File names

artifact	name
Analysis	`analysis-US-001.md`
Manual test cases	`cases-US-001.xlsx` + `.md` + `-coverage.md`
BDD test cases	`US-001-signup.feature` + `US-001-coverage.md`
Bug	`BUG-001.md`
Automation	`<type>/features/US-001—<slug>.feature` · `steps/US-001.steps.ts` · `locators/US-001.locators.ts`

Performance	`performance/k6/http/US-001–<slug>.ts` · `performance/artillery/US-001–<slug>.yaml` · `performance/jmeter/US-001–<slug>.jmx`
Mapping	`output/mappings/mapping-US-001–<type>.md`
Exploratory	`output/exploratory/<US or site>/exploratory-report.html` + `exploration.json` · `screenshots/` · `videos/` · `history/` (last 5)
Web performance	`output/web-performance/<US or site>/web-performance-report.html` + `web-performance.json` · `screenshots/` · `lighthouse/`
Failure analysis	`output/failures/<US or suite>/failures-report.html` + `failures.json` · `runs/` · `evidence/`
Coverage	`output/coverage/<US or all>/coverage-report.html` + `coverage.json`
Visual regression	`output/pixel-perfect/<US or site>/pixel-perfect-report.html` + `pixel-perfect.json` · `reference/` (kept) · `actual/` · `diffs/` · `crops/`
Cross-browser	`output/cross-browser/<US, site or suite>/cross-browser-report.html` + `cross-browser.json` · `screenshots/` · `suite/`
Repairs	`output/repairs/<US or suite>/repair-report.html` + `repairs.json` · `changes.diff` · `backup/`
Test case selection	`output/case-selection/<name>/selection-report.html` + `selection.json` · `selection.csv`
Accessibility	`output/accessibility/<US or site>/accessibility-report.html` + `audit.json` · `screenshots/` · `videos/` · `history/` (last 5)
Email	`output/emails/<US or mailbox>/emails-report.html` + `emails.json` · `screenshots/` · `eml/`
Translations	`output/translator/<US or site>/translator-report.html` + `translator.json` · `screenshots/` · `crops/`
Case review	`output/case-review/<US or name>/review-report.html` + `review.json`
Charters	`output/charters/<US or name>/charters.html` + `charters.md` · `sessions/CH-XX.md` · `sessions-report.html`
Usability	`output/ux/<US or site>/ux-report.html` + `ux.json` · `screenshots/` · `crops/`
Forms	`output/forms/<US or site>/forms-report.html` + `forms.json` · `screenshots/`
Logs	`output/logs/<name>/logs-report.html` + `logs.json`
History	`<each tool's folder>/history/<date-time>/` (last 5, with their evidence) · `tools/k0lmena/reports/<suite>/history/` for `npm test`

// reference

Repository structure

```
k0lmenIA/
├── CLAUDE.md                Context and standards (Claude Code always reads it)
├── ARCHITECTURE.md          How the repo grows
├── CONNECTORS.md            How to enable the MCP connectors
├── CHANGELOG.md             Changes in each version
├── LICENSE                  MIT license
├── .env.example             Variables template (copy to .env)
├── .mcp.json                MCP connectors (no secrets; enabled per settings)
├── package.json             Shortcuts to run k0lmena from the root (npm test…)
├── docs/                    This site, the PDF and the diagrams
├── tests/                   Script tests (pytest and node --test)
├── evals/                   Agent evals: cases with traps and checks
├── .github/workflows/       CI (tests), synthetic monitoring and evals (manual)
├── .claude/
│   ├── agents/              The 76 agents
│   ├── skills/              Context, design, performance guide, execution, k0lmena
│   └── settings.json        Enabled connectors and permissions
├── input/                   stories/ documentation/ api/ bugs/ emails/ logs/
├── output/                  What the agents generate
├── templates/              Bug, test case spreadsheet, coverage, estimated time
├── scripts/                 Test cases, estimated time, review, charters, logs, documentation
index, reports and doctor
│   ├── management/             Xray, QMetry (QTM4J), AIO Tests and test cases for Azure DevOps
│   └── mcp/                 MCP connector launchers (they read the .env)
└── tools/
    ├── newman/              Postman collections
    ├── zap/                 OWASP ZAP: web security (passive)
    ├── mailpit/             Mailpit: test mailbox for the emails
    └── k0lmena/
        ├── web/             features/ steps/ locators/ + framework
        ├── api/             features/ steps/ (generic steps)
        ├── mobile/          features/ steps/ locators/ support/ apps/
        ├── performance/     k6/http/, artillery/ and jmeter/
        ├── reports/         HTML reports
        ├── run-tests.js     npm test runner
        └── run-perf.js      npm run perf runner
```

// reference

Troubleshooting

What happens if the story is missing information?

I don't know what I'm missing to install or configure

An agent returns "I need you to confirm"

A new agent doesn't show up

The web suite fails without a URL or clicks say "outside of the viewport"

`npm test` runs 0 scenarios

A scenario never runs

"k6 not found"

"JMeter not found" or "JMeter needs Java"

`npm run perf` doesn't run a load profile from the agent

An Xray/QMetry/AIO folder shows up as `C:/Program Files/Git/…`

An MCP connector doesn't connect

Do the tests always check the database?

A Python library is missing

// reference

Extending the project

Adding an agent

01

Create the file

`.claude/agents/my-agent.md` with `name` and `description` frontmatter. The description is what Claude Code uses to decide when to invoke it.

02 **Write the body**

In English, like the rest: role, inputs, process, output and rules.

03 **Define the format**

If it generates its own format, add its template in `templates/` or its script in `scripts/`, and its folder in `output/`.

04 **Normalize the tables**

If it generates tables in a `.md`, run them through `scripts/format_tables.py`.

05 **If it needs an answer from the person**

Copy the "When you are missing information or a confirmation" section from any agent: subagents can't ask mid-task, so they return "I need you to confirm".

Skills, connectors and tools

Skill

A folder in `.claude/skills/` with its `SKILL.md`; it's loaded only when needed.

MCP connector

An entry in `.mcp.json`, without secrets; tokens are read from the `.env` by a script in `scripts/mcp/`.

Tool

A subfolder in `tools/` with its README.

Repo tests

`python -m pytest tests` and `npm run test:repo`; they run in CI (Ubuntu and Windows) on every change. Add a test when you change a script.

Regenerating this documentation

```
node docs/generate.js        # diagrams (PNG) and the PDF, with k0lmena's Playwright
```

The site is `docs/index.html` (published with GitHub Pages at underc0delabs.github.io/k0lmenIA); the PDF comes out of the same HTML with print styles, and the diagrams from `docs/diagrams/`.

License

MIT license — © 2026 **QARMY**. k0lmena is an open source project by **Danilo Vezzoni**. Made by the **Underc0de** community.